分类号 _____ 学校代码 10674

UDC _____ 密　　级 _____

昆明理工大学

硕士学位论文

（学术学位）

基于长度分区的
集合相似度查询算法研究

Research on Set Similarity Query Algorithm Based on Length Partition

研　究　生　姓　名 赵永雪

指导教师姓名、职称 贾连印 教授

学　科　专　业 计算机科学与技术

研　究　方　向 计算机系统结构

论文工作起止日期 2024 年 9 月~2026 年 6 月

论　文　提　交　日　期 2026 年 6 月

遵守学术行为规范承诺

本人已熟知并愿意自觉遵守《昆明理工大学研究生学术规范实施细则（试行）》的所有内容，承诺所提交的毕业和学位论文是终稿，不存在学术不端行为，且论文的纸质版与电子版内容完全一致。

独创性声明

本人声明所提交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得昆明理工大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。本人完全意识到本声明的法律结果由本人承担。

关于论文使用授权的说明

本人完全了解昆明理工大学有关保留使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。（保密的论文在解密后应遵守此规定）

本学位论文属于（必须在以下相应方框内打“√”，否则一律按“非保密论文”处理）：

1、保密论文：　☐本学位论文属于保密。

2、非保密论文：☐本学位论文属于内部论文，网上延后公开。

☐本学位论文不属于保密范围，适用本授权书。

是否同意授权以下单位（必须在以下相应方框内打“√”，否则一律按“同意授权”处理）：

☐同意授权　　☐不同意授权

将本人学位论文著作权中的数字化复制权、发行权、汇编权和信息网络传播权的专有使用权在全世界范围内授予中国学术期刊（光盘版）电子杂志社，并在《中国优秀博硕士学位论文全文数据库》和 CNKI 系列数据库中出版。

研究生本人签名：_____　　签字日期：20　　年　　月　　日

研究生导师签名：_____　　签字日期：20　　年　　月　　日

摘 要

集合数据广泛存在于文本检索、网页去重和数据清洗等应用场景中。集合相似度查询旨在从集合数据集中高效检索与给定查询集合满足相似度阈值的所有集合，是集合数据管理与分析中的关键问题之一，具有重要的应用价值。从计算机系统结构视角出发，设计高效的索引组织和查询处理机制，以支持海量集合数据上的实时相似度检索，对提升系统整体性能具有重要意义。

现有集合相似度查询方法仍存在若干不足。第一，多数方法未能充分利用集合长度信息进行索引组织与搜索空间约束，导致过滤粒度较粗、候选集规模较大。第二，在过滤阶段，剪枝能力有限者难以有效压缩候选集，而剪枝能力较强者需逐节点计算相似度上界或执行复杂的边界判断，导致过滤阶段本身开销较高。第三，在大规模数据集上，传统单一索引结构存在树高增加和节点访问增多等问题，制约了查询效率。针对上述问题，本文主要开展了以下工作：

1. 设计基于长度分区的多 B^+树索引（LMB）。该索引按照集合长度对数据集进行分区，将集合映射到一维整型键，并为每个分区独立构建 B^+子树，设计长度分区索引（LPI）用于高效管理 B^+子树。基于 LMB，实现基于对称差的集合相似度查询算法 LMB-SD。该算法首先利用长度过滤进行初步剪枝，基于对称差生成长度特定的键边界，避免逐节点计算相似度上界的复杂过程；继而通过基于对称差的键过滤策略对边界内的键进行深度筛选并生成候选集，最终执行精确验证。实验表明，LMB 索引构建时间较短、空间开销较低；LMB-SD 的查询效率优于对比算法，较 MBQ 提升高达 29.40 倍，较 $LeBQ^+$提升达 2.14 倍。

2. 针对 LMB-SD 生成的键边界仍较宽，候选键集规模较大的问题，设计基于相异差的集合相似度查询算法 LMB-AD。该算法将对称差拆分为两个相异的单向差，采用基于长度过滤和相异差的键边界计算策略生成更紧凑的键边界，有效减少候选键集的规模。在此基础上，进一步设计基于相异差的键过滤策略进行深度过滤，并引入基于最小覆盖和提前中止的验证策略，降低验证开销。实验结果显示，LMB-AD 的查询效率优于对比算法，较 $LeBQ^+$提升达 11.13 倍，较 LMB-SD 提升达 2.87 倍。本文提出的方法可有效支持各类可表示为集合的实体数据上的相似度检索任务，为高效相似度查询提供了具备实用价值的系统级解决方案。

关键词：集合相似度查询；长度分区；多 B^+树；对称差；相异差

Abstract

Set data widely exist in application scenarios such as text retrieval, web page deduplication, and data cleaning. Set similarity query aims to efficiently retrieve all sets from a dataset that satisfy a given similarity threshold with respect to a query set, which is one of the key problems in set data management and analysis and holds significant application value. From the perspective of computer system architecture, designing efficient indexing organization and query processing mechanisms to support real-time similarity retrieval on massive set data is of great importance for improving overall system performance.

Existing set similarity query methods still suffer from several limitations. First, most methods fail to fully exploit set length information for index organization and search space pruning, resulting in coarse filtering granularity and large candidate set sizes. Second, in the filtering stage, methods with limited pruning ability struggle to effectively reduce candidate sets, while methods with strong pruning ability require node-by-node similarity upper bound computation or complex boundary judgment, leading to high filtering overhead. Third, on large-scale datasets, traditional single-index structures suffer from increased tree height and node accesses, which restricts query efficiency. To address the above problems, this thesis mainly carries out the following work:

1. A length-partition-based multi B^+-tree index (LMB) is designed. This index partitions the dataset according to set length, maps each set into a one-dimensional integer key, independently builds a B^+-tree for each partition, and designs a Length Partition Index (LPI) to efficiently manage these B^+-trees. Based on LMB, a symmetric-difference-based set similarity query algorithm, LMB-SD, is implemented. The algorithm first uses length filtering for preliminary pruning, generates length-specific key boundaries based on symmetric difference to avoid the complex process of node-by-node similarity upper bound computation, then applies a symmetric-difference-based key filtering strategy to deeply filter keys within the boundaries and generate candidate sets, and finally performs exact verification. Experimental results show that LMB has low index construction time and space overhead, and LMB-SD achieves

significantly better query efficiency than compared algorithms, achieving up to 29.40× improvement over MBQ and 2.14× improvement over LeBQ$^+$.

2. To address the problem that LMB-SD still generates relatively wide key boundaries and large candidate key set sizes, an asymmetric-difference-based set similarity query algorithm, LMB-AD, is designed. This algorithm splits the symmetric difference into two asymmetric one-way differences, adopts a length-filtering and asymmetric-difference based key boundary calculation strategy to generate tighter key boundaries, and effectively reduces the candidate key set size. On this basis, an asymmetric-difference-based key filtering strategy is further designed for deep filtering, and a verification strategy based on minimum overlap and early termination is introduced to reduce verification overhead. Experimental results demonstrate that LMB-AD achieves better query efficiency than compared algorithms, with up to 11.13× improvement over LeBQ$^+$ and 2.87× improvement over LMB-SD. The proposed method can effectively support similarity retrieval tasks on various types of entity data that can be represented as sets, providing a practical system-level solution for efficient set similarity query.

Keywords: Set Similarity Query; Length Partition; Multi-B$^+$-tree; Symmetric Difference; Asymmetric Difference

目录

摘 要.....I
Abstract.....III
1 绪论.....1
1.1 研究背景及意义.....1
1.2 国内外研究现状.....2
1.2.1 集合相似度连接研究现状.....3
1.2.2 集合相似度查询研究现状.....4
1.3 论文出发点和主要工作.....6
1.4 论文组织结构.....8
1.5 本章小结.....9
2 相关理论和技术介绍.....11
2.1 相关定义.....11
2.1.1 集合数据集定义.....11
2.1.2 集合相似度查询定义.....11
2.1.3 符号定义.....13
2.2 常见的索引结构.....14
2.2.1 B^+树.....14
2.2.2 倒排索引.....15
2.2.3 R 树.....16
2.2.4 前缀树.....17
2.2.5 位图.....18
2.3 常见的过滤技术.....19
2.3.1 长度过滤.....19
2.3.2 分区过滤.....20
2.3.3 前缀过滤.....21
2.3.4 位置过滤.....22
2.3.5 后缀过滤.....22
2.4 本章小结.....23
3 基于长度分区和对称差的集合相似度查询算法.....25
3.1 引言.....25

3.2 算法总体框架.....26
3.3 索引构建.....27
3.3.1 长度分区.....27
3.3.2 键生成.....27
3.3.3 LMB 构建.....29
3.4 查询执行.....32
3.4.1 查询处理.....32
3.4.2 基于长度过滤和对称差的键边界计算.....33
3.4.3 基于对称差的键过滤.....36
3.4.4 验证.....37
3.5 复杂度分析.....39
3.5.1 空间复杂度.....39
3.5.2 时间复杂度.....40
3.6 实验分析.....40
3.6.1 数据集.....40
3.6.2 对比索引技术.....41
3.6.3 分区数对查询效率的影响.....42
3.6.4 过滤策略的效率.....43
3.6.5 对比其他算法.....44
3.6.6 扩展到其他相似度函数.....46
3.7 本章小结.....47
4 基于长度分区和相异差的集合相似度查询算法.....49
4.1 引言.....49
4.2 算法总体框架.....50
4.3 索引构建.....51
4.4 查询执行.....51
4.4.1 基于长度过滤和相异差的键边界计算.....51
4.4.2 基于相异差的键过滤.....52
4.4.3 基于最小覆盖和提前中止的验证.....53
4.5 复杂度分析.....55
4.5.1 空间复杂度.....55

4.5.2 时间复杂度.....55
4.6 实验分析.....56
4.6.1 分区数对查询效率的影响.....56
4.6.2 过滤策略的效率.....56
4.6.3 验证策略的有效性.....58
4.6.4 对比键边界计算策略.....59
4.6.5 对比其他算法.....59
4.6.6 扩展到其他相似度函数.....60
4.7 本章小结.....61
5 总结与展望.....63
5.1 本文主要工作总结.....63
5.2 本文进一步的研究方向.....64
致谢.....67
参考文献.....69
附录 A 攻读硕士学位期间的学术成果.....75
附录 B 攻读硕士学位期间参与的科研项目.....77

1 绪论

1.1 研究背景及意义

随着大数据与人工智能技术的快速发展，非结构化数据呈现出爆炸式增长态势。作为一种简洁且高效的数据表示方式，集合在数据建模与信息组织中发挥着重要作用。例如，在文本处理领域，一篇文档可以表示为词项集合；在电子商务场景中，用户的购物车可以表示为商品集合。大量现实对象经过特征抽取与离散化处理后，均可表示为集合数据，这对高效的集合数据处理技术提出了更高要求。

集合数据处理中的典型问题主要包括集合包含查询[1,2]与集合相似度计算两类。其中，集合包含问题主要研究集合之间的包含关系；集合相似度问题则关注如何度量不同集合之间的相似程度，并据此完成高效检索、匹配与分析。本文聚焦于集合相似度研究，重点讨论集合相似度连接与集合相似度查询两个关键问题。

集合相似度连接旨在从两个集合数据集中找出所有满足相似度阈值的相似集合对，给定两个集合数据集D_1和D_2，以及一个相似度阈值δ，其目标是查找所有的集合对$\langle S_1, S_2 \rangle$，使得集合S_1和S_2之间的相似度大于等于相似度阈值δ，其中，$S_1 \in D_1$，$S_2 \in D_2$。如果$D_1 = D_2$，则称为集合相似度自连接。由于集合相似度连接任务通常需要在大量集合对之间进行两两比较，直接计算代价较高，现有研究大多采用过滤-验证框架[3,4]，在过滤阶段，通过构建动态索引等过滤技术生成候选集，再在验证阶段得到最终结果。近年来，研究者已提出多种经典的集合相似度连接算法，为后续相关研究奠定了重要基础。

近年来，集合相似度查询问题受到越来越多学术界与工业界研究者的关注。集合相似度查询是指给定一个查询集合Q和一个相似度阈值δ，从数据集D中查找所有的集合$S \in D$，使得集合S和查询集合Q之间的相似度大于等于相似度阈值δ。虽然其目标与集合相似度连接相近，但二者在计算模式上存在显著差异：集合相似度连接动态构建索引并关注于一个固定的相似度阈值，而集合相似度查询通常在整个数据集上构建静态索引，并支持不同的相似度阈值下的高效检索。由于索引构建时间通常不计入查询时间，除了简单的倒排索引[5]，B^+树[6]、R 树[7]等复杂索引也被广泛应用于集合相似度查询，以提升查询阶段的执行性能。

集合相似度连接与查询算法在众多领域具有广泛的应用价值。在文本处理领域，可用于文本抄袭检测[8,9]与代码克隆检测[10-13]；在商业智能领域，可作为推荐

算法[14-17]、数据清洗与数据集成的重要技术支撑；在社交网络分析以及复杂对象检索任务中，可用于谣言检测、图相似度计算[18-22]以及大规模特征匹配等问题。相关研究已在上述方向取得了丰富成果，进一步体现了集合相似度计算的重要理论价值与应用潜力。

然而，随着数据规模的持续增长，现有集合相似度连接与查询算法在处理海量集合数据时仍面临诸多挑战。一方面，现有许多方法对集合长度信息利用不足，难以实现更细粒度的搜索空间缩减；另一方面，现有基于过滤-验证框架的方法在过滤阶段主要存在两类问题：一是部分方法过滤能力有限，难以有效压缩候选集规模；二是部分方法为了增强剪枝效果，需要逐节点计算相似度上界或执行复杂的边界判断，导致过滤阶段本身开销较高。此外，在大规模数据集上，传统单一索引结构容易面临树高增加、节点访问增多等问题，从而影响整体查询效率。上述问题制约了集合相似度连接与查询算法在实际场景中的应用效果与效率。因此，围绕高效索引结构设计与过滤策略优化开展研究，对于提升集合相似度连接与查询效率、降低计算资源消耗，具有重要的理论意义和实际应用价值。

从计算机系统结构的视角来看，集合相似度查询的性能瓶颈主要体现在存储访问模式、索引组织方式以及查询处理流程的设计上。本文以系统结构优化的思想为指导，通过设计基于长度分区的多 B^+树索引结构、优化键边界计算策略以及引入高效的过滤验证机制，从数据组织、索引层次和查询执行路径三个层面提升系统整体查询效率。这一研究思路体现了计算机系统结构专业“面向特定应用负载优化系统设计”的核心思想。

1.2 国内外研究现状

集合相似度研究主要包括集合相似度连接和集合相似度查询两个方向。集合相似度连接旨在从两个数据集中找出满足相似度阈值的所有相似集合对；集合相似度查询则是在给定查询集合和相似度阈值的条件下，从数据集中检索与查询集合满足相似度阈值的结果集合。二者虽然都以集合间相似关系为研究对象，但在计算模式上存在明显差异：集合相似度连接通常面向固定的相似度阈值进行批量处理，多采用动态索引与过滤-验证框架；集合相似度查询则更强调面向整个数据集构建静态索引，并支持不同相似度阈值下的高效在线检索。总体来看，集合相似度连接的研究已较为成熟，而集合相似度查询在索引组织和候选控制等方面仍有较大的研究空间。基于此，本文从集合相似度连接和集合相似度查询两个层

面，对国内外研究现状进行梳理。

1.2.1 集合相似度连接研究现状

集合相似度连接旨在从两个数据集中找出满足相似度阈值的所有相似集合对，相关研究起步较早，已形成较为成熟的技术体系。由于传统方法需要在大量集合对之间进行两两验证，直接计算代价极高，现有研究大多采用经典的过滤-验证框架，即先利用高效过滤策略生成尽可能少的候选集合对，再在验证阶段对候选结果进行精确判断。如何通过有效的索引组织和过滤规则尽可能缩小候选集合对规模，是提高集合相似度连接效率的关键。

索引作为一种常见的过滤手段，通过构建专门的数据结构，将原始集合数据组织为便于搜索和剪枝的索引形式，从而避免对整个数据集进行线性扫描，因此其被广泛应用在集合相似度连接任务中。例如，Qi 等人提出了聚合倒排索引，可同时支持字符串连接与分数属性剪枝[23]；Sandes 等人利用哈希函数将集合映射为定长位图，并结合 GPU 加速完成集合相似度连接[24]；近年来，频繁模式树（Frequent Pattern Tree，FP-tree）也被应用于连接任务，其中，FastTELP-SJ[25]通过 FP-tree 挖掘频繁二项集从而快速计算集合交集，在低阈值场景下具有较好的连接效率；文献[26]进一步将 FP-tree 与 MapReduce 结合，用于大规模集合相似度自连接。总之，通过索引结构可以减少候选生成开销，但其性能往往依赖于数据分布特征以及索引构建方式。

在索引的基础上，一些高效的过滤策略被广泛应用在集合相似度连接任务中，进一步过滤掉不可能满足相似度阈值的集合对。其核心思想是在精确验证之前，利用相似度约束设计高效的过滤规则，尽可能减少候选集合对的规模。其中，前缀过滤是集合相似度连接中最具代表性的技术路线之一。例如，Wang 等人提出了自适应长度的前缀过滤框架 Adapt[27]，通过成本模型为不同集合选择合适的前缀长度，从而降低候选生成开销。随后，后缀过滤和位置过滤等过滤方法也被引入到连接任务中，用于在前缀过滤基础上进一步剔除不可能满足相似度阈值的候选集合对。例如，PPJoin 和 PPJoin^{+}[28]通过综合利用元素位置与后缀信息有效减少了候选集合对规模，从而提高了连接效率。可以看出，集合相似度连接研究已逐步从单一过滤规则发展为多阶段协同剪枝机制。

随着数据规模和计算平台的发展，并行集合相似度连接方法也得到了广泛研究。例如，MG-Join[29]和 FS-Join[30]分别通过整体过滤和分段过滤支持大规模并行处理；Bellas 等人对图像处理器（Graphics Processing Unit，GPU）环境下的集合

相似度连接技术进行了系统评估[31]；Gallet 等人从负载均衡角度优化了 GPU 进而加速相似度连接[32]；Feng 等人提出了基于 GPU 的文档余弦相似度自连接算法[33]。除了并行计算外，一些研究者还从近似计算方面提出了多种扩展方法，其中大多是基于局部敏感哈希（Locality Sensitive Hashing，LSH）的连接方法，通过保持相似对象的哈希邻近性来快速定位候选集合对，牺牲部分精确性换取更高的处理效率。例如，Yu 等人通过引入代表点记录组减少了 LSH 的查找次数[34]；Christiani 等人实现了可指定召回率的 LSH 连接方法[35]；Sohrabi 等人则在超大规模数据集上实现了基于 LSH 的并行集合相似度连接[36]。此外，面向安全场景的加密数据连接问题也受到关注，文献[37]研究了细粒度安全约束下的加密数据连接。上述工作表明，集合相似度连接研究已逐步从精确处理扩展到并行计算、近似计算和安全查询等更复杂的场景。

总体来看，集合相似度连接已经形成了较完善的研究体系，在候选生成和过滤剪枝等方面积累了丰富成果，为集合相似度查询研究奠定了重要基础。然而，集合相似度连接方法通常面向固定的相似度阈值和批量离线处理，其动态索引和候选生成策略通常服务于一次性连接过程，难以直接适应静态索引条件下支持任意相似度阈值的在线查询任务。因此，虽然连接研究为集合相似度查询提供了重要理论基础和技术借鉴，但查询问题仍然具有独立而重要的研究价值。鉴于此，本文主要研究集合相似度查询问题。

1.2.2 集合相似度查询研究现状

集合相似度查询是在给定查询集合和相似度阈值的条件下，从数据集中检索与查询集合满足相似度阈值的结果集合，它更关注给定查询集合下的快速检索能力，其核心目标是在数据集上构建高效静态索引，并在不同相似度阈值条件下尽可能减少查询时的搜索代价。集合相似度查询算法已广泛应用于数据清理[38,39]、剽窃检测[40]、对象检测[41]和数据集成[42]等场景。由于索引构建时间通常不计入查询开销，研究者往往采用比连接任务更复杂的索引结构和更细粒度的过滤策略，以提升集合相似度查询性能。借鉴集合相似度连接，现有的集合相似度查询方法通常采用过滤-验证框架[43]。近年来，研究者们结合索引结构和过滤策略进行协同过滤，提出了多种高效的集合相似度查询方法。

倒排索引因结构简单，候选生成直接，在集合相似度查询中得到了广泛应用。例如，MergeOpt[44]通过在倒排索引中记录元素所在集合及其位置信息来加快候选集生成；Cong 等人[45]和 Li 等人[46]结合了倒排文件与树结构，用于文本检索和

空间邻近查询。基于倒排索引的方法在候选生成阶段具有较高效率，尤其适用于基于元素重叠关系的快速筛选，但当高频元素较多或倒排列表过长时，候选集规模仍可能较大，从而影响整体查询性能。

树也是一种被广泛应用于集合相似度查询中的索引结构。例如，Zhang 等人提出了基于 B^+树的集合相似度查询算法 MBQ[47]，利用 k-means 聚类将集合分区，并将集合映射为键，在每个分区上构建独立 B^+树。在查询期间，MBQ 从根节点开始自顶向下逐节点调用 Upper 函数，计算查询集合与每个节点的相似度上界。如果该相似度上界小于给定的相似度阈值，则以该节点为根的子树中的所有集合都不可能与查询集合满足相似度阈值，因此可以直接修剪该子树；文献[7]提出的 DualTrans 算法则将集合映射为固定长度的向量，并基于 R 树执行范围搜索，从而将集合相似度查询转化为空间搜索问题。在查询期间，DualTrans 自顶向下地计算每个节点的最小外包矩形与查询集合的距离下界（类似于 MBQ 中的相似度上界）。上述两种方法的共同特点是借助层次化索引逐步缩小搜索范围，但在查询过程中仍需进行逐节点相似度上界或距离下界计算，因此存在较高的节点级计算开销。此外，它们未能充分利用集合长度信息进行索引组织和搜索空间约束，这导致过滤粒度较粗、候选集规模较大，后续验证的计算开销也随之增加，从而降低了查询效率。LeBQ 和 LeBQ$^+$[6]在整个数据集上构建全局 B^+树索引，能够支持高效范围检索，但在处理大规模数据集时，单一全局索引结构容易导致索引高度增加，从而带来更高的搜索开销；文献[48]基于 B^+树设计了支持加密的相似度查询方法，进一步扩展了树结构索引在安全场景下的应用。在处理大量共享前缀的数据集合时，前缀树也表现出较好的适用性。例如，Jia 等人提出的 ETI 索引与 T-Similarity 算法[49]将 T-Overlap 查询映射为前缀树上的 T-节点查询，避免了过滤-验证框架中的候选生成与逐一验证。进一步地，Jia 等人[50]提出了 T-starTrie 索引和 TT-SSQ 算法，将具有相同前缀的集合组织到同一节点，并通过一次节点级过滤同时裁剪该节点对应的多个集合，从而集合相似度查询问题可以转化为 T-starTrie 上的第一层匹配节点检测问题，显著提升了候选生成效率。

与此同时，位图索引及相关轻量级表示方法也被用于集合相似度查询。Li 等人提出 LES3[51]，使用深度学习方法对数据集进行分组，尽可能将具有更多共同元素的集合分配到同一组，并利用轻量级位图结构支持高效搜索。除此之外，Chauhan 等人实现了基于布隆过滤器的并行集合相似度查询方法[52]；Deng 等人提出基于动态规划的前缀选择方法[53]，通过挑选具有较高剪枝能力的前缀元素进

一步提升过滤效果；文献[54]研究了动态集合下的自适应过滤方法；文献[55,56]研究了高效且保护隐私的集合相似度查询；文献[57]研究了语义集合相似度查询；文献[58]研究了多核系统下的并行集合相似度查询。

综上，现有集合相似度查询方法主要从索引组织和过滤剪枝等方面开展研究。尽管这些方法在不同场景下取得了良好效果，但仍普遍存在集合长度信息利用不足、搜索边界不够紧凑、过滤阶段代价较高以及大规模数据集下单一索引效率受限等问题。尤其对于精确的集合相似度查询，如何在充分利用集合长度信息的基础上进一步优化索引组织与候选控制，仍是值得深入研究的问题。

1.3 论文出发点和主要工作

本文聚焦于精确的集合相似度查询问题。结合前述研究现状可以看出，现有研究虽然为集合相似度查询提供了丰富的理论和技术基础，但仍存在以下三个方面的不足：

（1）集合长度信息利用不足

现有算法普遍未能充分利用集合长度信息进行索引组织和搜索空间约束，这导致过滤粒度较粗、候选集规模较大，后续验证的计算开销也随之增加，从而降低了查询效率。例如，对于长度差异较大的集合，即使它们仅包含少量公共元素，也可能被保留为候选对象，从而造成不必要的计算开销。

（2）过滤能力不足或过滤代价较高

现有集合相似度查询方法大多基于过滤-验证框架，其查询性能在很大程度上取决于过滤阶段对候选集的控制能力。然而，现有方法采用的过滤策略主要存在两类问题：一类方法过滤能力有限，难以有效压缩候选集规模，从而增加后续验证开销；另一类方法虽然具有较强的剪枝能力，但往往需要执行逐节点相似度上界计算、复杂边界判断或多阶段过滤过程，使过滤阶段本身带来较高的时间开销。例如，MBQ 和 DualTrans 虽然采用了树结构剪枝策略，但其逐节点计算相似度上界或距离下界的方式仍会对整体查询性能造成影响。

（3）大规模数据集的挑战

随着科学与技术的迅速发展，数据规模持续增长。对于大规模数据集，传统单一索引结构（如单棵 B^+树）容易面临树高增加、节点访问增多等问题，从而导致查询延迟增加。如何在大规模数据集上支持高效的、精确的集合相似度查询，仍然是一个具有挑战性的研究问题。

针对上述的问题，本文在现有研究成果的基础上开展了如下工作：

（1）为充分利用集合长度信息进行索引组织并降低传统单一索引结构在大规模数据集上的查询开销，本文首先设计了一种基于长度分区的多 B^+树索引结构 LMB。该索引首先按照集合长度对数据集进行分区，将具有相同长度的集合划分到同一分区中；随后设计了一种键生成方法，通过分区映射将集合表示为向量，并顺序连接向量中的分量值生成一维整型键；在此基础上，为每个分区分别构建独立的 B^+树，从而形成面向整个数据集的 B^+树森林结构，显著降低了索引高度并提升了查询效率；为高效管理这些索引子树，本文进一步设计了长度分区索引 LPI，可直接获取特定长度对应的索引子树。基于 LMB 索引，本文设计了一种基于对称差的集合相似度查询算法，LMB-SD。针对传统方法未充分利用集合长度信息加速查询和过滤效率较低的问题，本文提出了一种基于长度过滤和对称差的键边界计算策略，能够针对特定长度快速生成对应的键边界，从而将集合相似度查询问题转换为多个范围查询子问题。此外，本文还设计了一种基于对称差的键过滤策略，用于进一步排除位于键边界范围内但与查询集合不满足相似度阈值的键，从而有效减少候选集规模。实验结果表明，LMB 具有较短的索引构建时间、较低的索引空间开销和较小的索引高度。LMB-SD 的查询性能显著优于对比方法，较 MBQ 最高提升了 29.40 倍，较 LeBQ$^+$最高提升了 2.14 倍。

（2）尽管 LMB-SD 能够高效计算键边界，但其生成的键边界仍相对宽泛，候选键集的规模较大，从而增加了后续过滤与验证开销。为进一步压缩搜索范围，本文在 LMB 索引基础上提出了一种基于相异差的集合相似度查询算法 LMB-AD。该算法将对称差拆分为两个相异的单向差，并设计了一种基于长度过滤和相异差的键边界计算策略，从而生成更加紧凑的键边界。相较于 LMB-SD，LMB-AD 可将键边界缩小近一半，有效减少了候选键集的规模。在此基础上，本文进一步设计了一种基于相异差的键过滤策略用于深度过滤，并引入基于最小覆盖和提前中止的验证策略，以降低后续验证开销。实验结果表明，LMB-AD 的查询性能优于对比方法，较 LeBQ$^+$最高提升了 11.13 倍，较 LMB-SD 最高提升了 2.87 倍。

此外，针对大规模数据集下单一索引结构效率受限的问题，LMB 通过长度分区将全局索引分解为多个独立的局部 B^+子树，显著降低了索引高度和节点访问次数，进而提升查询效率。实验表明，在千万级数据集 TWEETS 上，LMB 的最大 B^+索引高度为 5，而传统全局 B^+树（例如，LeBQ$^+$算法）高度为 6；LMB 的平均 B^+索引高度低至 0.93，并且，LMB-SD 的查询效率较 LeBQ$^+$提升达 2.14 倍，

验证了所提方法在大规模场景下的有效性。

本文研究成果可在以下典型场景中发挥重要作用：文本抄袭检测：将文档表示为词项集合，通过集合相似度查询快速定位疑似抄袭片段；电商平台商品去重：将商品属性表示为集合，识别重复或高度相似的商品信息；社交网络分析：基于用户兴趣标签集合，实现相似用户群体挖掘与推荐；生物信息学：处理基因序列、蛋白质结构等集合化数据，辅助相似性比对与分析。因此，本文所提算法不仅具有理论价值，更具备面向实际应用部署的潜力。

1.4 论文组织结构

本文共分为五章，各章内容安排如下：

第一章为绪论。本章主要介绍论文的研究背景与意义，梳理集合相似度连接与集合相似度查询的国内外研究现状，分析现有方法在索引组织、过滤策略和查询效率等方面取得的进展与存在的不足。在此基础上，明确本文的研究出发点，概括介绍本文提出的基于长度分区的多 B^+树索引 LMB、基于对称差的集合相似度查询算法 LMB-SD 以及基于相异差的集合相似度查询算法 LMB-AD，并给出论文的整体结构安排。

第二章为相关理论与技术介绍。本章首先给出集合数据集、集合长度信息、集合相似度查询问题及相关相似度度量的基本定义；随后介绍 B^+树等常用的索引结构，以及长度过滤等常用的过滤技术；最后对本文后续方法设计所涉及的关键理论基础进行说明。

第三章为基于长度分区和对称差的集合相似度查询算法。本章首先分析现有集合相似度查询方法在长度信息利用、过滤能力和查询效率等方面存在的不足；在此基础上，提出了一种基于长度分区的多 B^+树索引 LMB，以及一种基于对称差的集合相似度查询算法 LMB-SD；最后通过实验对所提索引和算法的有效性进行验证与分析。

第四章为基于长度分区和相异差的集合相似度查询算法。本章在第三章研究的基础上，进一步分析 LMB-SD 在键边界范围和候选控制方面的局限，提出了一种基于相异差的集合相似度查询算法 LMB-AD；同时结合相应的键过滤与验证优化策略，并通过实验结果验证该方法的有效性和性能优势。

第五章为总结与展望。本章对全文的研究工作进行总结，归纳本文的主要研究结论，并结合集合相似度查询领域的发展趋势，对后续研究方向进行展望。

1.5 本章小结

本章围绕本文的研究内容，对集合相似度的研究背景、研究现状以及论文的研究出发点和主要工作进行了系统阐述。首先，结合大数据时代海量非结构化数据的处理需求，说明了集合表示方式的普遍性及集合相似度研究的重要应用价值；其次，从集合相似度连接和集合相似度查询两个层面对国内外相关研究进行了梳理；随后，针对现有集合相似度查询方法在长度信息利用不足、过滤能力有限以及查询效率不高等方面存在的问题，明确了本文的研究出发点，并给出了相应的解决思路；在此基础上，概括介绍了本文提出的基于长度分区的多 B^+树索引 LMB、基于对称差的集合相似度查询算法 LMB-SD 和基于相异差的集合相似度查询算法 LMB-AD，为后续章节的展开奠定了基础。

2 相关理论和技术介绍

2.1 相关定义

为了便于后续方法的描述与分析，本节首先对本文涉及的基本概念和相关定义进行说明，包括集合数据集、集合长度信息、相似度度量以及集合相似度查询问题等内容。这些定义和符号不仅构成了后续索引结构设计与查询算法推导的基础，也为后文实验分析中的相关讨论提供了统一表述。

2.1.1 集合数据集定义

由多个集合S构成的集合族记为集合数据集D，数据集D中的所有集合数量记为$|D|$。若某个元素e至少出现在一个集合S中，则称e为数据集D中的一个元素。将数据集D中所有出现过的元素去重后所形成的元素集合，称为数据集D的不同元素集合，记为$U=\{e_1,e_2,\ldots,e_n\}$。对于任意元素e，若其出现在多个集合中，则在U中仅计数一次。因此，每个集合S均为U的一个子集。集合S中元素的个数称为集合的长度，也称为集合的基数，记为$|S|$。

本文中提到的“集合长度信息”不仅包括描述集合规模的基数信息，通常记为$|S|$，还包括在相似性约束下进一步推导得到的候选集合长度范围。在集合相似度查询中，集合长度信息可用于限定候选集合的可能长度区间，从而有效缩小检索空间并提高查询效率。基于这一性质，本文在索引构建阶段依据集合长度对数据集进行划分，并在各长度分区上建立独立的B^+树索引；在查询阶段，首先利用长度过滤缩减候选范围，随后结合长度信息设计了长度特定的键边界计算策略，进而将集合相似度查询转化为多个范围查询子问题。

在本文中，每个集合S均为普通集合而非多重集，即集合S中不包含重复的元素，并且每个集合均具有唯一的集合标识符ID。本文还使用Q表示查询集合，以区别于数据集D中的集合对象S。表 2.1 给出了一个示例集合数据集，其中，第一列为集合标识符ID，第二列为集合中的具体元素。

2.1.2 集合相似度查询定义

两个集合之间的相似度可以通过 Jaccard，Cosine 和 Dice 等度量函数来计算。具体而言，给定两个集合S_1和S_2，Jaccard 度量函数的计算公式为：

$$Jaccard(S_1,S_2)=\frac{|S_1\cap S_2|}{|S_1\cup S_2|} \tag{2.1}$$

其中，$S_1 \cap S_2$ 表示集合 S_1 和 S_2 的交集，$S_1 \cup S_2$ 表示集合 S_1 和 S_2 的并集，$|S|$ 表示集合 S 中元素的个数，即集合 S 的基数。

表 2.1 示例集合数据集

ID	集合
S_1	$\{e_6, e_7, e_{10}, e_{12}, e_{13}\}$
S_2	$\{e_6, e_8, e_{11}, e_{16}, e_{17}, e_{18}\}$
S_3	$\{e_1, e_2, e_3, e_4, e_5, e_{11}\}$
S_4	$\{e_1, e_9, e_{10}, e_{14}, e_{15}, e_{17}, e_{20}\}$
S_5	$\{e_1, e_2, e_6, e_7, e_{11}, e_{12}, e_{16}\}$
S_6	$\{e_1, e_2, e_6, e_7, e_{11}, e_{12}, e_{16}, e_{18}, e_{19}\}$
S_7	$\{e_1, e_3, e_5, e_8, e_{11}, e_{13}, e_{15}, e_{16}, e_{17}, e_{18}\}$
S_8	$\{e_1, e_3, e_4, e_6, e_{12}, e_{13}, e_{14}, e_{16}, e_{17}, e_{19}\}$
S_9	$\{e_1, e_2, e_6, e_7, e_8, e_9, e_{11}, e_{16}, e_{17}, e_{18}, e_{19}\}$
S_{10}	$\{e_1, e_3, e_4, e_6, e_7, e_8, e_9, e_{11}, e_{12}, e_{13}, e_{14}, e_{17}\}$

Cosine 度量函数的计算公式为：

$$\mathrm{Cos}ine(S_1, S_2) = \frac{|S_1 \cap S_2|}{\sqrt{|S_1| * |S_2|}} \tag{2.2}$$

Dice 度量函数的计算公式为：

$$Dice(S_1, S_2) = \frac{2|S_1 \cap S_2|}{|S_1| + |S_2|} \tag{2.3}$$

由于 Jaccard 相似度能够同时反映集合交集与并集之间的关系，且在集合相似度查询研究中应用最为广泛，因此本文采用 Jaccard 相似度作为集合间的相似度度量，但本文提出的方法可以扩展到其他度量函数。基于此，定义 1 给出了集合相似度查询的正式定义。

定义 1（集合相似度查询） 给定一个查询集合 Q 和一个相似度阈值 δ，集合相似度查询的目标是在数据集 D 中查找所有集合 $S \in D$，使得 S 和 Q 之间的相似度大于等于 δ，即需要满足条件 $Sim(S, Q) \geq \delta$，其中，$Sim()$ 是一个相似度函数。

示例 2.1： 根据表 2.1 给出的示例集合数据集，给定一个查询集合 $Q = \{e_1, e_2, e_6, e_7, e_{11}, e_{12}, e_{16}, e_{20}\}$ 和相似度阈值 $\delta = 0.7$，执行集合相似度查询将返回 S_5 和 S_6，因为 $Jaccard(S_5, Q) = 7/8 = 0.875$，$Jaccard(S_6, Q) = 7/10 = 0.7$ 都不小于

相似度阈值δ。在本文中，若集合S和查询集合Q之间的相似度大于等于相似度阈值δ，则称S和Q相似，否则称S和Q不相似。

2.1.3 符号定义

为了提高文章的可读性和规范性，本文采用的符号解释详见表 2.2。

表 2.2 符号解释表

符号	描述
D	集合数据集
U	数据集D中的不同元素集合
S	数据集D中的一个集合
e	元素
ID	集合的唯一标识符
Q	查询集合
δ	相似度阈值
M	分区数量
V	分区向量
K	整型键
L	集合长度
L_-	长度下界
L_+	长度上界
$\lambda_{S,Q}$	Q和S的对称差
$\Lambda_{S,Q}$	Q和S的对称差上界
$\Lambda_{L,Q}$	Q长度特定L的对称差上界
$\eta_{S,Q}$	Q和S的上界差
$\nabla_{S,Q}$	Q和S的上界差上界
$\nabla_{L,Q}$	Q长度特定L的上界差上界
$\mu_{S,Q}$	Q和S的下界差
$\Delta_{S,Q}$	Q和S的下界差上界
$\Delta_{L,Q}$	Q长度特定L的下界差上界
$K_{L,-}$	Q长度特定L的键下界
$K_{L,+}$	Q长度特定L的键上界

2.2 常见的索引结构

为提高集合相似度查询效率，索引结构被广泛应用于候选集生成和搜索空间缩减过程中。合理的索引结构能够将原始集合数据组织为便于检索和剪枝的形式，从而避免对整个数据集进行线性扫描，降低查询开销。在集合相似度查询中，常见的索引结构包括倒排索引、树结构索引以及位图索引等。基于此，本节对几类常见的索引结构进行介绍。

2.2.1 B^+树

B^+树[59]是 B 树的一种变形结构，是一种经典的多路平衡搜索树，广泛应用于数据库索引和操作系统文件系统中。B^+树具有结构稳定、层次清晰、查找效率高以及范围检索性能好等优点，其插入、删除和查找操作通常都具有较稳定的对数时间复杂度。B^+树的内部节点不直接存储实际数据记录，而仅用于存储索引键并引导搜索方向；所有数据记录均存储在叶子节点中，且叶子节点通常通过顺序指针连接形成有序链表。B 树与 B^+树的主要区别如表 2.3 所示。

表 2.3 B^+树与 B 树的主要区别

特性	B 树	B^+树
数据存储位置	所有节点都可能存储数据	仅叶子节点存储数据
叶子节点连接	叶子节点之间没有链接	叶子节点形成有序链表
内部节点的功能	既存储索引也存储数据	仅作为索引使用
关键字重复	关键字不重复	内部节点关键字会重复

本文采用常见的 B^+树定义方式，即内部节点中的关键字个数比孩子节点个数少 1。对于阶数为 f 的 B^+树，其内部节点至多包含 f 个孩子节点和 $f-1$ 个关键字。并且，内部节点中的关键字按照从小到大的顺序排列，多个孩子指针将关键字空间划分为若干有序区间，每个孩子节点对应其中一个关键字范围。叶子节点中的记录同样按照关键字大小有序排列，叶子节点之间通过顺序指针连接形成有序链表。图 2.1 给出了一棵阶数为 4 的 B^+树示例。

相对于 B 树，B^+树在存储效率方面具有明显优势。由于内部节点不存储实际数据记录，在相同节点大小条件下，B^+树可以容纳更多的关键字和分支，从而降低树的高度，减少查询时所需访问的节点数量。与 B 树不同，B^+树的查找操作总是从根节点开始，最终到达叶子节点；即使在内部节点中遇到与目标关键字相等的索引项，也仍需继续沿相应指针向下搜索，直到定位到叶子节点中的目标数

据记录。因此，B^+树的查询路径长度较为稳定，有利于提高检索性能。

此外，由于所有叶子节点按照关键字顺序组织并通过链表相连，B^+树能够方便地支持范围查询和顺序访问。在集合相似度查询任务中，若将集合映射为有序键值，则可以利用 B^+树高效执行基于键范围的候选搜索，从而缩小检索范围并降低查询开销。因此，B^+树是一种非常适合用于集合相似度查询索引构建的基础结构。鉴于 B^+树在查询方面的显著优势，本文采用 B^+树作为主要的索引结构，通过在整个数据集上构建 B^+森林，降低索引高度的同时加速集合相似度查询。

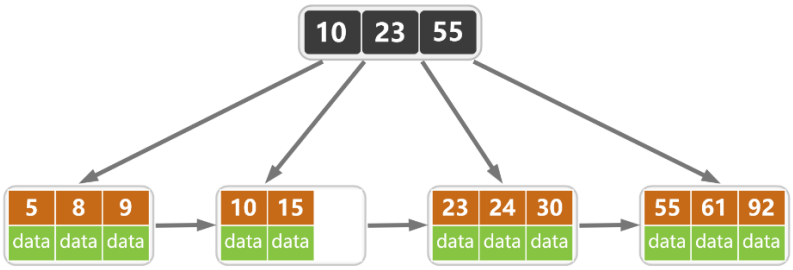


图 2.1 B^+树索引结构

2.2.2 倒排索引

倒排索引[60]是一种经典且高效的索引结构，广泛应用于文本检索、信息检索以及相似度查询等任务中。其基本思想是：以元素或关键字为索引项，为每个索引项维护一个倒排列表，用于记录包含该元素或关键字的对象标识符。与顺序扫描整个数据集相比，倒排索引能够直接定位包含特定元素或关键字的对象集合，从而显著缩小搜索范围，提高查询效率。

倒排索引通常由目录和倒排列表两部分组成。目录中存储数据集中所有不同的元素或关键字，每个元素或关键字在目录中对应一个倒排列表；倒排列表中记录了包含该元素或关键字的对象标识符，必要时还可进一步存储元素在对象中的位置信息、出现频次或其他辅助属性。对于集合数据而言，倒排索引可以看作是“元素到集合”的反向映射关系，即通过某个元素快速找到所有包含该元素的集合。现假设样本数据集为 $S_0=\{a,b,c,d\}$，$S_1=\{a,d,f\}$，$S_2=\{a,b,d,e\}$，$S_3=\{a,b,c,f,g\}$，$S_4=\{b,c,d,e\}$，则可据此建立元素与集合标识符的倒排索引结构，如图 2.2 所示。

在集合相似度查询中，倒排索引具有实现简单、候选生成效率高等优点。给定查询集合后，可以通过访问查询集合中各元素对应的倒排列表，快速获得可能与查询集合相似的候选集合，并结合重叠计数和位置过滤等策略进一步缩小候选

范围。因此，倒排索引是集合相似度查询中最常用的基础索引结构之一。

然而，倒排索引也存在一定局限性。当数据规模较大或高频元素较多时，部分倒排列表可能非常长，从而增加候选集生成与后续验证的开销；同时，倒排索引主要依据元素重叠关系进行检索，对集合长度信息和更复杂的结构特征利用不足。因此，在实际应用中，倒排索引常与其他过滤策略或复合索引结构结合使用，以进一步提高查询性能。

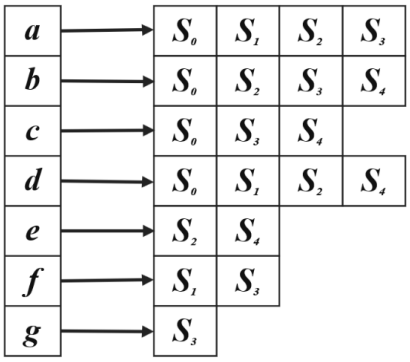


图 2.2 倒排索引结构

2.2.3 R 树

R 树[61]是一种经典的空间索引结构，是一种高度平衡的多路树，主要用于多维空间对象的组织与检索。R 树中的叶子节点存储实际的空间对象及其对应的最小外包矩形（Minimum Bounding Rectangle，MBR），内部节点则保存其孩子节点的外接矩形及相应指针，通常以（Rectangle，Child）的形式组织，其中，Rectangle 反映了该节点下所有孩子节点所覆盖的区域，Child 则指向具体的孩子节点，R 树索引结构如图 2.3 所示。

R 树可看作是 B 树在多维空间上的推广，它通过层次化组织空间对象的包围区域，实现对多维数据的高效检索。基于最小外包矩形的组织方式，R 树能够支持点查询、范围查询和近邻查询等多种检索操作。查询过程中，系统只需访问与目标区域相交或相关的少量中间节点，即可逐步缩小搜索范围，从而提高检索效率。

R 树还具有较好的动态更新能力，能够支持对象的插入、删除和查询操作，因此在空间数据库、地理信息系统以及多维相似性检索等场景中得到了广泛应用。在集合相似度查询研究中，若将集合映射为固定长度向量，则可利用 R 树对向量空间进行层次化组织，并通过范围搜索快速生成候选对象。因此，R 树也是集合相似度查询中一种具有代表性的索引结构。

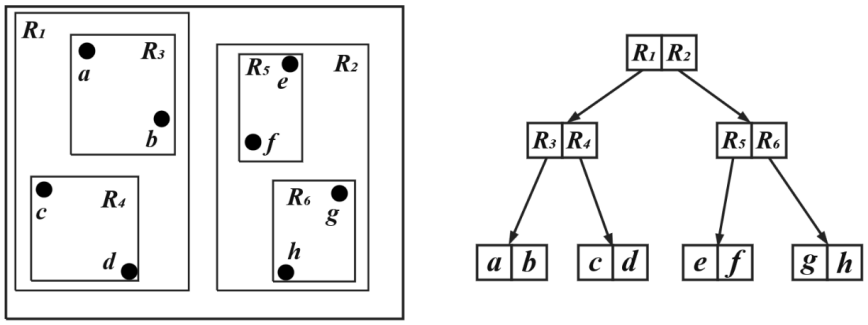


图 2.3 R 树索引结构

2.2.4 前缀树

前缀树[62]又称为字典树，通常简记为 Trie。Trie 是一种基于公共前缀共享思想构建的树形索引结构，最初主要用于字符串匹配与检索任务。其基本思想是按照字符串或序列中元素的顺序逐层构建树结构，其中，每个节点表示一个字符或一个元素，不同对象中相同的前缀部分可在树中共享同一路径，从而避免重复存储相同前缀。由于具有良好的前缀组织能力，Trie 特别适用于处理大量具有公共前缀的数据对象。

Trie 的结构通常由根节点和多层子节点组成。根节点本身不对应具体字符或元素，其每条向下路径表示一个字符串或序列的前缀扩展过程；叶节点或终止标记则用于表示完整对象的结束。通过从根节点逐层向下遍历，Trie 能够高效完成精确匹配、前缀匹配以及相关对象的定位。Trie 的索引结构如图 2.4 所示。

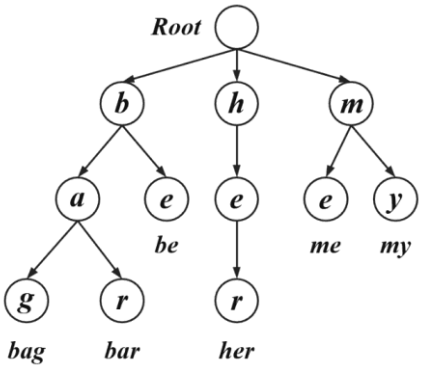


图 2.4 前缀树索引结构

与倒排索引主要建立“元素到对象”的映射关系不同，Trie 更强调对对象内部顺序结构和公共前缀的组织能力。当数据对象中存在大量共享前缀时，Trie 能够通过路径复用显著降低冗余存储开销，并提高前缀相关查询的执行效率。因此，在需要根据公共前缀快速定位候选对象的场景中，Trie 具有较好的适用性。

在集合相似度查询研究中，若将集合中的元素按照统一顺序组织，则前缀树也可用于集合对象的层次化表示与搜索。例如，对于共享前缀较多的数据集合，Trie 能够通过前缀共享减少索引冗余，并支持基于前缀约束的候选生成。相关研究表明，前缀树在处理 T-Overlap 查询等问题时具有较好的检索性能。

2.2.5 位图

位图索引[63]是一种将数据对象映射为二进制向量的索引结构，广泛应用于数据库检索、集合运算和相似度计算等任务中。其基本思想是：以数据对象的属性值或元素出现情况为依据，使用二进制位对对象进行编码，其中，某一位取值为 1 表示对应属性值或元素存在，取值为 0 则表示不存在。通过这种方式，位图索引能够将数据对象表示为紧凑的二进制形式，从而支持高效的按位运算。

在查询过程中，位图索引可以利用按位与、按位或等运算快速判断数据对象之间的重叠关系或包含关系，因此在需要频繁执行集合交、并等操作的场景中具有较高效率。对于集合数据而言，若将数据集中的所有不同的元素映射到位图的不同位置，则每个集合都可以表示为一个二进制向量。例如，现假设样本数据集为 $S_0=\{a,b,c,d\}$，$S_1=\{a,d,f\}$，$S_2=\{a,b,d,e\}$，$S_3=\{a,b,c,f,g\}$，$S_4=\{b,c,d,e\}$，则可将不同元素映射到位图的不同位置，构建一个简单的位图索引，如表 2.4 所示。这样，在计算集合之间的交集、并集或重叠程度时，可通过位运算快速完成，从而减少传统逐元素比较带来的开销。

位图索引具有结构简单、运算高效和易于并行化等优点，特别适合处理维度相对固定、位运算频繁的数据场景。然而，当数据集规模较大或数据稀疏性较强时，位图长度可能显著增加，从而带来较高的存储开销；同时，不同映射方式和压缩策略也会影响其实际检索性能。因此，在集合相似度查询中，位图索引通常作为辅助索引或过滤工具，与其他索引结构结合使用，以进一步提升查询效率。

表 2.4 位图索引结构示例

集合	二进制向量
$S_0=\{a,b,c,d\}$	[1,1,1,1,0,0,0]
$S_1=\{a,d,f\}$	[1,0,0,1,0,1,0]
$S_2=\{a,b,d,e\}$	[1,1,0,1,1,0,0]
$S_3=\{a,b,c,f,g\}$	[1,1,1,0,0,1,1]
$S_4=\{b,c,d,e\}$	[0,1,1,1,1,0,0]

2.3 常见的过滤技术

为了提高集合相似度查询效率，除了构建高效索引结构外，过滤技术也是减少搜索空间、缩小候选集规模的重要手段。其基本思想是在保证查询结果正确性的前提下，利用集合之间的长度关系、前缀重叠关系以及元素位置信息等特征，尽可能提前排除不可能满足相似度阈值的集合，从而降低后续精确验证的计算开销。在集合相似度查询中，常见的过滤技术主要包括长度过滤、分区过滤、前缀过滤、位置过滤和后缀过滤等。基于此，本节对几类常见的过滤技术进行介绍。

2.3.1 长度过滤

长度过滤[64]的核心思想基于集合长度与相似度度量之间的约束关系。在海量数据环境下，若直接计算查询集合与数据集中每个集合之间的相似度，往往需要较高的计算代价；相比之下，获取集合长度信息的开销极低。长度过滤正是利用查询集合与候选集合之间的长度差异，提前排除那些由于长度差异过大而不可能满足相似度阈值的集合。该策略仅需访问集合的长度信息，而无需读取集合的具体元素内容，因此具有较低的时间复杂度。

以最常用的 Jaccard 相似度为例，假设查询集合为Q，其长度为$|Q|$，数据集中某一候选集合为S，其长度为$|S|$，给定的相似度阈值为δ。根据 Jaccard 相似度的定义，若集合S和查询集合Q相似，则需要满足$Jaccard(S,Q)\geq\delta$。进一步，可以推出集合S的长度必须满足不等式约束：$|Q|*\delta\leq|S|\leq|Q|/\delta$。这意味着，对于给定的查询集合$Q$和相似度阈值$\delta$，所有长度落在上述区间之外的集合都不可能与查询集合Q满足相似度要求，因此可在后续元素比较之前直接过滤。类似地，在 Cosine 相似度和 Dice 相似度等常用度量下，也存在相应的长度约束关系，因此长度过滤具有较好的普适性。

尽管长度过滤的理论基础已经较为成熟，但在现有许多集合相似度查询方法中，长度过滤往往未得到充分利用。部分方法仅将长度过滤作为一种辅助性的后处理手段，在索引构建阶段并未显式考虑长度特征，导致长度差异较大的集合仍被组织在同一索引结构中，从而增加了搜索深度和无效访问开销。基于这一点，本文后续方法进一步将长度信息融入索引组织过程，通过长度分区将不同长度的集合进行有效划分，并在查询过程中，首先结合长度过滤进行初步过滤，结合长度信息生成了长度特定的键边界，缩小候选范围。因此，本文充分利用了集合长度信息进行索引组织和搜索空间约束，有利于提升查询效率。

2.3.2 分区过滤

分区过滤[65]是一种通过合理划分数据集来降低搜索复杂度的有效剪枝策略，其核心思想源于“分而治之”的原则，即根据数据对象的某些特征或规则，将整个数据集划分为若干相对独立的分区。在查询过程中，算法首先依据查询对象的特征快速定位可能包含相似结果的目标分区，而将不相关分区直接排除在搜索范围之外。通过这种方式，原本面向全局数据集的搜索过程被缩小为局部分区内的搜索过程，从而减少了不必要的计算开销，并有助于提升查询算法在大规模数据环境下的可扩展性。此外，合理设计的分区策略还能够有效利用数据本身的分布特性，将相似度较高的对象聚集在同一分区内，从而进一步提高后续过滤和验证的效率。

在集合相似度查询中，分区过滤的关键在于分区键的选择，这直接决定了分区的剪枝能力和检索效率。常见的分区方式包括基于哈希的分区、基于数据分布的分区以及基于特征属性的分区等。例如，基于哈希的分区利用哈希函数将对象映射到不同桶中，查询时仅需访问相关桶；基于特征属性的分区则利用对象的固有属性进行划分，如集合长度、元素特征或其他统计信息。近年来，也有研究者提出了基于机器学习和深度学习的分区策略。理想的分区策略应使同一分区内的对象具有较高的相似潜力，而不同分区之间的对象相似潜力较低，从而提高分区过滤的有效性。同时，分区设计还应保证查询结果的完整性，即所有满足相似度条件的对象不会因分区策略而被遗漏。另外，分区边界的定义方式也会影响查询性能：过宽的分区边界会引入较多无关对象，过窄的边界则可能导致相似对象被误排除。因此，分区策略需要在剪枝能力和查询完整性之间取得平衡。

尽管分区过滤在提升查询效率方面具有明显优势，但现有部分方法在分区策略上仍存在进一步优化空间。例如，一些方法采用较为简单的哈希分区，未充分考虑集合数据的内在分布特征，容易导致分区负载不均衡；另一些方法的分区粒度较粗，使得单个分区内的数据量仍然较大，从而削弱了分区剪枝效果。对于集合相似度查询而言，若能选择具有较强区分能力且计算开销较低的特征作为分区依据，则有助于进一步缩小搜索范围，提高查询效率。基于上述思想，本文后续方法进一步将集合长度这一重要特征引入分区索引构建过程，通过长度分区将不同长度的集合划分到相对独立的索引子结构中。该策略不仅能够高效地进行长度过滤，还能够通过分区组织减少无效搜索空间，更重要的是，可以为后续长度特定的键边界查询提供支持。

2.3.3 前缀过滤

前缀过滤[27]是一种经典且常用的剪枝策略，其核心思想建立在集合元素重叠约束之上。根据集合相似度度量的定义，若两个集合之间的相似度达到给定的相似度阈值，则它们之间必然具有一定数量的公共元素。前缀过滤正是利用这一性质，从每个集合中选取一部分具有代表性的前缀元素，并基于“若两个集合相似，则其前缀必然存在重叠”这一条件进行候选筛选。通过这种方式，原本需要对集合全部元素进行比较的问题，被转化为对少量前缀元素的匹配问题，从而有效降低了相似度计算的开销。

前缀过滤的关键在于前缀长度的确定。一般而言，在给定相似度阈值的条件下，可根据不同相似度度量函数推导出满足查询完整性要求的最小公共元素约束，并据此确定集合需要保留的前缀长度。换言之，若两个集合满足给定相似度阈值，则它们的前缀部分至少应包含一个公共元素；否则这两个集合不可能满足相似度要求，可直接予以剪枝。该思想不仅适用于 Jaccard 相似度，也可扩展到 Dice、Cosine 等常见的集合相似度度量函数。给定一个查询集合 Q，其前缀长度的计算公式为 $|Q|-\min Over(Q)+1$，其中，$\min Over(Q)$ 表示查询集合 Q 的最小覆盖。对于 Jaccard、Dice、Cosine、海明距离等相似度度量函数，它们的最小覆盖分别为 $\delta|Q|$，$\delta|Q|/(2-\delta)$，$\delta^2|Q|$，$|Q|-\delta$。

在实际实现中，前缀过滤通常结合全局元素排序和倒排索引来完成。首先，对数据集中所有不同的元素按照全局出现频率进行排序，通常采用从低频到高频的顺序，以增强低频元素的区分能力；随后，按照该全局元素顺序对数据集中每个集合内部元素重新排列，并计算前缀长度和截取其前缀部分作为索引特征。在构建倒排索引时，以前缀部分作为索引项，记录包含该前缀部分的集合标识符列表。查询时，仅需提取查询集合的前缀部分，并访问对应的倒排列表即可生成候选集。由于低频元素通常具有更强的区分能力，这种基于频率排序的前缀策略能够有效减少高频元素带来的无效候选生成。

尽管前缀过滤能够显著缩小搜索空间，但在大规模数据集上仍存在一定局限。一方面，前缀重叠仅能说明两个集合“可能相似”，却无法保证其最终相似度满足阈值，因此生成的候选集中往往包含较多伪候选，增加了后续验证开销；另一方面，当数据规模较大或高频元素较多时，倒排列表可能过长，从而使列表访问和合并成本成为性能瓶颈。此外，前缀过滤主要依赖元素重叠信息，对集合长度这一强约束特征利用不足，因此在长度差异较大的数据场景下，过滤效果往往受

到限制。

2.3.4 位置过滤

位置过滤[66]是一种在前缀过滤基础上进一步利用元素位置信息进行剪枝的策略。其基本思想是：对于满足相似度阈值的两个有序集合而言，公共元素不仅需要出现在前缀中，其在各自集合中的位置也应满足相应的约束关系。因此，在根据前缀过滤生成候选集之后，可以根据匹配元素的位置关系进一步排除不可能满足相似度要求的候选对象，从而提高过滤精度。

在实际实现中，位置过滤通常依赖于带有位置信息的倒排索引。与传统倒排索引仅记录对象标识符不同，增强型倒排索引还需记录元素在集合中的位置。当查询集合与候选集合在某个前缀元素上匹配时，可结合相似度阈值推导出的位置信息约束，判断该候选集合是否仍可能满足相似度要求。若所有匹配元素的位置均无法满足约束条件，则该候选集合可直接过滤，而无需进入后续完整验证阶段。

位置过滤能够在一定程度上减少伪候选，提高候选筛选的精度，是前缀过滤的重要补充策略。然而，该方法需要额外存储位置索引，并在过滤过程中执行位置比对，因此也会带来一定的时间和空间开销。

2.3.5 后缀过滤

后缀过滤[67]是一种用于优化候选集验证过程的剪枝技术，通常作为前缀过滤之后的进一步补充策略使用。其提出的背景在于：尽管前缀过滤能够有效缩小候选集规模，但由前缀重叠生成的候选集合中往往仍包含大量伪候选，即仅凭前缀重叠尚不能判定其最终满足相似度阈值。若对这些候选对象直接进行完整的元素比对，仍会带来较高的计算开销。后缀过滤正是利用集合中非前缀部分的剩余信息，在最终精确验证之前进一步剪枝，从而减少不必要的后续比较。

后缀过滤的核心思想是：在已知前缀部分匹配情况的基础上，估计两个集合后缀部分所能提供的最大重叠贡献，从而判断该候选对象是否仍有可能满足相似度阈值。通常在全局元素排序之后，集合被划分为前缀和后缀两部分，其中，前缀用于候选生成，后缀用于进一步验证。对于某个候选集合，若在当前已匹配元素的基础上，即使假设其后缀部分全部匹配，也无法使总体相似度达到给定阈值，则可提前终止验证过程，并将该候选对象直接剪除。通过这种方式，后缀过滤能够有效避免对后缀部分的无效扫描。

后缀过滤能够在一定程度上降低候选验证的计算代价，是前缀过滤后的重要

补充策略。然而，其本质上仍依赖于逐元素或逐位置的动态判断过程，涉及较多的边界估计和增量计算。当集合长度较长或数据规模较大时，这类计算本身也可能带来额外开销。

2.4 本章小结

本章围绕集合相似度查询的理论基础与关键技术进行了系统介绍，为后续索引和算法的设计与实现奠定了基础。首先，对集合数据集、集合相似度度量、集合相似度查询问题及相关符号进行了形式化定义，明确了本文研究对象、基本概念及统一的数学表示。其次，介绍了 B^+树、倒排索引、R 树、前缀树和位图索引等常见的索引结构，并分析了各类索引在数据组织方式、检索效率和适用场景等方面的特点。结合集合相似度查询的需求以及本文后续方法设计，B^+树因具备良好的平衡性、高效的范围检索能力以及较低的实现复杂度，被作为后续索引构建的重要基础。最后，本章对常见的过滤技术进行了梳理，重点介绍了长度过滤、分区过滤、前缀过滤、位置过滤和后缀过滤等方法，并分析了它们在缩小搜索空间和减少候选集规模方面的作用。其中，长度过滤和分区过滤与本文研究关系最为密切，二者分别从长度约束和搜索空间划分两个角度为后续索引构建及查询算法设计提供了技术支撑。

3 基于长度分区和对称差的集合相似度查询算法

3.1 引言

尽管现有集合相似度查询方法在索引组织与过滤剪枝方面取得了一定进展，但仍存在若干不足。首先，许多方法未能充分利用集合长度信息进行索引组织和搜索空间约束，导致过滤粒度较粗、候选集规模较大，从而影响查询效率。其次，部分基于树结构的查询方法需要逐节点计算相似度上界或执行复杂的边界判断过程，带来了较高的查询开销。例如，MBQ 在查询过程中需要逐节点计算相似度上界，并涉及位提取、键分割等操作，查询过程较为复杂。此外，LeBQ$^+$在整个数据集上构建单一全局 B$^+$树索引，在大规模数据集场景下容易面临索引高度增加、节点访问增多等问题，从而进一步影响查询性能。

索引结构的选择直接影响集合相似度查询的整体效率。常见的索引结构包括倒排索引、R 树、前缀树和 B$^+$树等。其中，倒排索引虽能快速定位包含特定元素的集合，但在高频元素场景下倒排列表过长，候选集规模难以控制；R 树适用于多维空间数据，但将集合映射为向量后会引入信息损失与边界膨胀问题；前缀树适合处理共享前缀较多的数据，但对集合内部元素顺序敏感，通用性受限。相比之下，B$^+$树具有结构稳定、范围检索高效、动态更新支持良好等优点，且通过将集合映射为一维整型键，可直接利用 B$^+$树的有序性实现高效的键边界检索。因此，本章选取 B$^+$树作为基础索引结构。

本章围绕“充分利用集合长度信息、降低索引搜索代价、减少候选集规模”这一目标展开研究。为此，本章首先设计了一种基于长度分区的多 B$^+$树索引结构 LMB，通过将不同长度的集合划分到不同分区并分别构建独立的 B$^+$子树，降低索引高度并提高索引组织效率。通过将整个数据集组织为多个独立的局部索引，从而降低单一索引在大规模场景下的搜索开销。在此基础上，进一步提出了一种基于对称差的集合相似度查询算法 LMB-SD。该算法通过基于长度过滤和对称差的键边界计算策略，能够快速生成长度特定的键边界，从而避免传统方法中逐节点计算相似度上界的复杂过程；同时，结合基于对称差的键过滤策略，进一步减少候选集规模，提升查询效率。

本章后续内容安排如下：3.2 节给出算法的总体框架；3.3 节介绍基于长度分区的多 B$^+$树索引结构 LMB，包括长度分区、键生成和 LMB 构建三个部分；3.4

节详细阐述基于对称差的集合相似度查询算法 LMB-SD，包括查询处理、键边界计算、键过滤和验证四个主要内容；3.5 节进行空间复杂度和时间复杂度分析；3.6 节通过实验分析验证所提索引结构与查询算法的有效性和优越性。

3.2 算法总体框架

本章提出的 LMB-SD 算法整体遵循索引构建与查询执行两阶段架构，如图 3.1 所示。

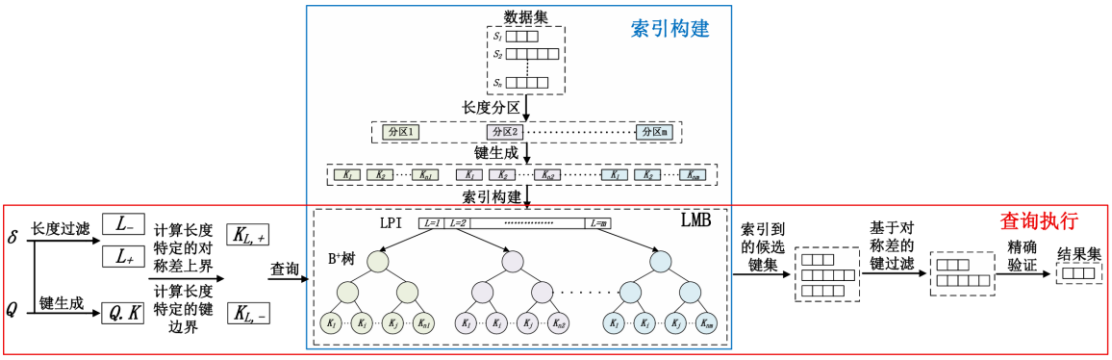


图 3.1 LMB-SD 算法总体框架

索引构建阶段：如图 3.1 中的蓝色边框区域所示，首先，采用基于集合长度的分区策略，将数据集中具有相同长度的集合划分至同一分区（长度分区）。其次，将每个集合转换为一维整型键（键生成）。最后，为每个长度分区分别构建独立的 B⁺树索引，形成 B⁺树森林，并由长度分区索引（Length Partition Index，LPI）统一管理各 B⁺子树。该索引结构将全局数据划分为多个局部索引，有效降低单一索引在大规模数据集上的搜索高度与节点访问开销。

查询执行阶段：如图 3.1 中的红色边框区域所示，对于给定的查询集合 Q 和相似度阈值 δ，首先计算 Q 的一维整型键 $Q.K$（键生成），并计算其长度下界 L_- 与长度上界 L_+，筛选出满足相似度阈值的候选长度分区（长度过滤）。随后，对于每个候选长度 L，计算 Q 长度特定的对称差上界，并通过键边界计算策略生成长度特定的键下界与键上界，从而将集合相似度查询转换为多个范围查询子问题。在键边界范围内，算法依次访问各 B⁺子树中的键，采用基于对称差的键过滤策略对键进行深度筛选，排除不可能相似的键及其对应的所有集合，生成候选集。最后，对候选集中的集合执行精确的 Jaccard 相似度验证，得到最终结果集。

上述框架通过长度分区实现了搜索空间的初步缩减，利用基于对称差的键边界计算策略避免了传统方法中逐节点计算相似度上界的高昂开销，并结合键过滤与精确验证保证了查询结果的正确性与效率。

3.3 索引构建

为充分利用集合长度信息进行索引组织并降低传统单一索引结构在大规模数据集上的查询开销，本文设计了一种基于长度分区的多 B^+树索引结构 LMB。该索引首先按照集合长度对数据集进行划分，再为每个长度分区分别构建独立的B^+树，从而形成面向整个数据集的 B^+树森林结构。并且，本文设计长度分区索引（LPI）高效管理这些 B^+子树。通过这种方式，LMB 不仅能够减少跨长度范围的无效搜索，还能够降低索引的高度，提升整体检索效率。LMB 的构建过程主要包括长度分区、键生成和索引构建三个关键步骤，下面将分别进行介绍。

3.3.1 长度分区

为充分利用集合长度信息进行索引组织并降低无效搜索开销，LMB 首先采用基于集合长度的分区策略，将长度相同的集合划分到同一分区中，不同长度的集合则划分到不同分区。该策略建立在长度过滤的基础上：对于 Jaccard 相似度等常见度量，若两个集合满足给定的相似度阈值，则其长度必须满足相应的约束范围。因此，在索引构建阶段预先完成长度分区，可以在查询阶段直接过滤掉大量长度明显不满足条件的集合，从而减少候选对象数量。

此外，长度分区还为后续索引构建提供了更有利的数据组织形式。由于每个分区内的集合长度一致，分区内部的数据分布更加规整，便于后续将集合映射为一维整型键并分别构建独立的 B^+树子索引。通过这种方式，LMB 将原本面向全局数据集的单一索引转化为多个面向局部分区的子索引，不仅降低了索引高度，也减少了查询过程中的无效节点访问，从而提高了整体检索效率。

3.3.2 键生成

由于 B^+树主要适用于一维数据的索引，而集合是多维的数据，且集合的维度通常是不固定，难以直接利用 B^+树进行组织和检索。因此，本章首先将集合映射到一维整型键。具体而言，本章采用分区映射的方法，其核心思想是将数据集中所有不同元素 $e_i \in U$ 划分到 M 个分区中。为了尽可能平衡各分区中的元素数量及其累积频率，本章采用如下贪心分配策略：

（1）将 U 中的所有元素按其出现频率降序排序；

（2）从出现频率最高的第一个元素开始，依次将当前元素分配到累积频率最小的分区中，并更新该分区的累积频率；重复上述过程，直到所有元素分配完成，且每个元素仅属于一个分区。

示例 3.1：根据表 2.1 的示例集合数据集，本章首先统计各元素在数据集中的出现频率，并按频率从高到低对所有元素进行排序，结果如表 3.1 所示。假设分区数 $M=4$，即将所有元素划分到 4 个分区。出现频率最高的前 4 个元素（e_1,e_6,e_{11},e_{16}）分别被分配到分区 1、分区 2、分区 3 和分区 4。随后，继续处理其余元素，并始终将当前元素分配到累积频率最小的分区中。例如，下一个元素 e_{17}（出现频率为 6）被分配到分区 4（由于此时分区 4 的累积频率最小，其值为 6），并更新分区 4 的累积频率为 6+6=12。重复上述过程，直到所有元素完成分配，最终得到如表 3.2 所示的分配结果。

表 3.1 元素频率表

频率	元素
8	e_1
7	e_6,e_{11}
6	e_{16},e_{17}
5	e_7,e_{12}
4	$e_2,e_3,e_8,e_{13},e_{18}$
3	e_4,e_9,e_{14},e_{19}
2	e_5,e_{10},e_{15}
1	e_{20}

表 3.2 元素分配表

分区	元素	累积频率
1	e_1,e_2,e_3,e_4,e_5	21
2	e_6,e_7,e_8,e_9,e_{10}	21
3	$e_{11},e_{12},e_{13},e_{14},e_{15}$	21
4	$e_{16},e_{17},e_{18},e_{19},e_{20}$	20

每个元素仅属于一个唯一的分区。基于此，数据集中的任意一个集合 S 可以被分配到 M 个分区，生成一个 M 维向量 $V=[v_1,v_2,\ldots,v_M]$，本文将其称为分区向量。其中，第 i 维的分量值 v_i 表示集合 S 中的元素被分配到第 i 个分区的数量。

为了将向量 V 映射到一维整型键 K，本章将 V 中的 M 个分量依次编码并顺序连接。对于每个分量 v_i，本章使用 $b=\lfloor 64/M \rfloor$ 位二进制编码表示。由于可能会出现极端情况，集合中的所有元素都映射到同一个分区，所以本文设 LMB 索引能够支持的最大集合长度为 2^b-1。对于长度超过 2^b-1 的集合，本文将其称为长

集合，直接将它们与查询集合Q进行验证，而无需生成一维整型键。

示例 3.2：假设$M=4$，$b=4$，根据表 3.2 中的元素分配结果，集合$S_1=\{e_6,e_7,e_{10},e_{12},e_{13}\}$分配到 4 个分区的元素分别是∅，$\{e_6,e_7,e_{10}\}$，$\{e_{12},e_{13}\}$和∅。因此，可以得到分区向量$S_1.V=[0,3,2,0]$。然后，编码并顺序连接这 4 个分量得到集合$S_1$对应的一维整型键$S_1.K=0000001100100000$。对于表 2.1 的示例集合数据集，各集合生成的向量表示及其对应整型键如表 3.3 所示。需要说明的是，不同集合可能生成相同的向量表示。例如，S_7和S_8是两个不同的集合，但其对应的分区向量均为[3,1,3,3]，因此会映射为相同的一维整型键。

由于整型键K是向量V中的分量值的简单编码和顺序连接，当上下文明确时，本文使用$S.V$或$S.K$来表示集合S生成的一维整型键，并用$S.v_i$或$K.v_i$表示向量$S.V$中的分量值v_i。

表 3.3 示例数据集合生成的分区向量和整型键

ID	分区向量	整型键
S_1	[0,3,2,0]	0000001100100000
S_2	[0,2,1,3]	0000001000010011
S_3	[5,0,1,0]	0101000000010000
S_4	[1,2,2,2]	0001001000100010
S_5	[2,2,2,1]	0010001000100001
S_6	[2,2,2,3]	0010001000100011
S_7	[3,1,3,3]	0011000100110011
S_8	[3,1,3,3]	0011000100110011
S_9	[2,4,1,4]	0010010000010100
S_{10}	[3,4,4,1]	0011010001000001

3.3.3 LMB 构建

本章充分挖掘集合长度信息在集合相似度查询中的作用，提出了一种基于长度分区的索引组织策略。具体而言，具有相同长度的集合被划分到同一个长度分区中，而不同长度的集合则存储在不同的分区中，从而实现了数据的有序化和模块化。

在 LMB 索引中，每个长度分区均需支持高效的范围查询与动态更新。综合考虑多种索引结构的特性，B^+树具有以下优势：（1）平衡性好，查询、插入、删

除操作的时间复杂度均为对数级别，性能稳定；（2）范围检索高效，叶子节点通过链表连接，支持快速的区间扫描，与本文的键边界查询需求高度契合；（3）动态更新友好，无需像静态索引那样定期重建，便于数据集的增量维护；（4）实现成熟，存储效率高且易于扩展至外存环境。因此，本文选用 B+树作为各分区的底层索引结构。

本章设计了 LMB 索引来管理集合生成的一维整型键。具体而言，对于每一个长度 L，本章为所有长度为 L 的集合构建一棵独立的 B+树索引，记为 B_L。因此，针对整个数据集，LMB 实际上构建的是由多棵 B+树组成的 B+森林。该设计一方面能够充分利用长度过滤，将查询限制在可能满足长度约束的若干子树中；另一方面，也避免了在整个数据集上构建单一全局 B+树所带来的树高增加、节点访问增加以及查询效率下降等问题。

将 B_L 中的内部节点支持的最大的孩子节点数记为 f。因此，每个内部节点最多存储 f 个子节点指针和 $f-1$ 个键。此外，B_L 中所有的叶节点都位于同一层，并用指针连接以支持高效的范围查询。需要说明的是，由于不同集合可能生成相同的一维整型键，本章在叶节点中为每个键 K 附加一个数组 $K.Vec$，用于存储生成键 K 的所有集合的 ID，从而支持后续的快速过滤和验证。

为了高效管理 B+森林，LMB 在 B+森林之上引入了长度分区索引（Length Partition Index，LPI），从而形成二级索引结构。由于 LMB 支持的最大集合长度为 2^b-1，LPI 设计为一个长度为 2^b-1 的数组，其中，第 L 个位置存储指向 B_L 根节点的指针。该设计具有以下优点：

（1）能够通过 LPI 索引直接定位长度为 L 的子树 B_L，效率较高，从而减少索引访问开销。

（2）当长度为 L 的集合不存在时，传统全局 B+树通常仍需沿根到叶路径进行搜索后才能确认，而在 LMB 中，可直接通过 LPI 判断 B_L 是否为空，从而避免不必要的节点访问。

考虑到 LMB 中的每个 B_L 本质上仍是一棵 B+树，因此 B_L 同 B+树一样，支持高效的插入、删除等动态更新操作。与传统 B+树更新过程不同的是，除了需对树结构中的节点进行插入、删除或调整外，还需同步维护叶节点中每个键 K 所对应的附加数组 $K.Vec$。该数组用于记录所有映射到键 K 的集合标识符，因此在集合插入或删除时，需要相应地更新其内容。在索引动态更新过程中，数据集中元素的全局出现频率可能发生变化，从而使当前元素分区结果不再是基于最新频率统

计下的最优划分。然而，只要每个元素所属的分区保持不变，则由该分区映射生成的集合键值也保持不变。因此，全局频率的改变不会影响索引结构及查询结果的正确性，只可能对分区均衡性和索引性能产生一定影响。由于 B^+树更新机制已有较为成熟的研究，本文不再对其标准更新过程作详细展开。本文默认 LMB 索引驻留于内存中，但其结构同样可以较容易扩展到外部存储环境。

传统 B^+树的构建方法通常采用逐数据插入的方式，这可能导致大量的节点分裂和树结构调整，从而增加索引构建开销。为此，本章采用一种块构建的方法（https://github.com/wesleyburlani/b-plus-tree/blob/master/src/hpp/B.cpp），以提高 B^+树的构建效率，并使构建后的 B^+树结构更加均匀，层次更低，从而有利于后续查询。其基本思想是：将数据升序排序，然后尽可能将叶节点装满，自下而上地构建 B^+树。具体而言，首先按照一维整型键对所有集合进行升序排序，然后将排序后的键值按块分配给叶节点，构建叶节点层。然后，自下向上逐层生成内部节点，直至完成整棵 B^+树的构建。需要说明的是，在本文提出的键生成策略下，不同的集合可能生成相同的一维整型键。因此，即使采用块构建方法，仍然需要在构建阶段中扫描每个键 K，并将所有映射到该键的集合 ID 插入对应的 $K.Vec$ 中。尽管如此，块构建方法能够有效避免逐数据插入过程中频繁发生的节点分裂和结构调整，从而显著提升索引构建效率。

示例 3.3：假设 $f=3$（即每个节点可以存储两个数据键），对于表 2.1 中的示例集合数据集，本章构建的 LMB 索引如图 3.2 所示。为了便于展示，图中的键均采用向量形式表示。由图 3.2 可知，LMB 中的各 B_L 子树的平均高度仅为 1。相比之下，如果在整个数据集上构建一棵全局 B^+树索引，由于表 2.1 中的示例集合数据集共生成了 9 个键，故索引高度至少为 $\log_3 9+1=3$。因此，LMB 能够有效降低单棵索引树的高度，从而减少查询过程中的节点访问开销。特别地，当 L=8 时，数据集中并不存在长度为 8 的集合，此时，LMB 可通过 LPI 直接判断对应的子树 B_8 为空，而无需像传统全局 B^+树那样继续沿树搜索，从而避免了不必要的节点访问。

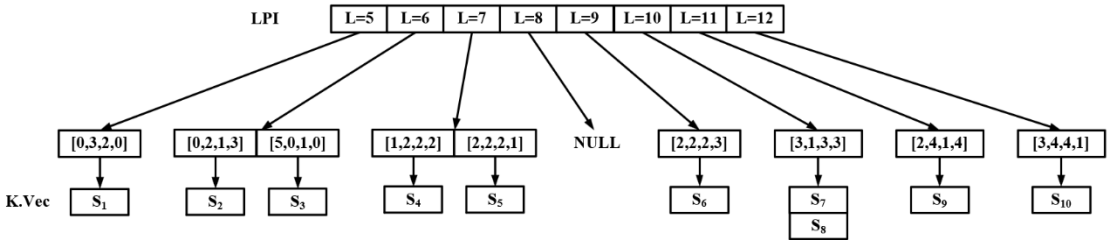


图 3.2 LMB 索引结构示例

3.4 查询执行

基于 LMB 索引的结构特点，本章进一步设计了基于对称差的集合相似度查询算法（LMB-SD）。与传统方法在索引遍历过程中逐节点计算相似度上界的方式不同，LMB-SD 利用相似度约束和长度过滤，推导出长度特定的键边界解析公式。通过直接计算键边界，LMB-SD 能够在索引遍历之前就确定搜索范围，从而将复杂的相似度计算转化为轻量级的数值比较，减少查询过程中的计算开销。LMB-SD 的整体流程主要包括以下四个部分：

（1）查询处理：作为算法的入口，该部分负责接收查询集合并提取其相关特征。基于 LMB 的长度分区机制，算法首先确定满足长度约束的目标分区，并排除不可能满足条件的无关分区，从而在全局范围内缩小后续检索空间。并针对满足长度约束，但不在 LMB 中索引的长集合，将它们直接与查询集合进行验证。

（2）键边界计算：该部分基于长度过滤和对称差原理，结合查询集合特征与相似度阈值，推导出特定长度下的目标键的上界和下界。与传统 B^+树索引中通过逐节点计算上界实现剪枝的方式相比，该方法能够直接给出键值搜索范围，从而减少不必要的计算操作。

（3）键过滤：在已知键边界的基础上，该部分利用 B^+树的有序性执行高效的范围查询，仅访问边界范围内的键值区间，并结合基于对称差的键过滤策略进一步剔除不满足条件的候选集合。

（4）验证：该部分对经过前序过滤得到的候选集合进行精确相似度验证，以保证查询结果的正确性。由于前续阶段已经缩小了候选范围，验证阶段的计算代价能够得到有效控制，从而在保证查询结果准确性的同时，提高整体查询效率。

3.4.1 查询处理

本章首先利用长度过滤剔除不可能与查询集合相似的候选集合。对于给定查询集合Q，若数据集D中某个集合S与查询集合Q的 Jaccard 相似度不小于相似度阈值δ，则集合S的长度必须落在一定的范围内；若其长度过大或过小，则不可能满足给定相似度约束。本文采用 Jaccard 相似度，因此，$L_- = \lceil |Q| * \delta \rceil$称为长度下界，$L_+ = \lfloor |Q| / \delta \rfloor$称为长度上界。长度在$[L_-, L_+]$之外的集合不可能和查询集合$Q$相似，可在查询初期直接过滤。

由于 LMB 仅索引了长度不超过$2^b - 1$的集合，与查询集合Q满足相似度阈值δ的候选长度范围可以分为以下三种情况：

（1）$L_- > 2^b - 1$，此时，与查询集合Q相似的集合均为长集合，即其长度全部超过 LMB 的可索引范围。对于这种情况，直接验证长度位于区间$[L_-, L_+]$内的每个集合S和查询集合Q的相似度。

（2）$L_- \le 2^b - 1 < L_+$，此时，候选长度区间一部分落在 LMB 的可索引范围内，另一部分超出该范围。对于区间$[L_-, 2^b - 1]$内的集合，可利用 LMB 执行查询并生成候选集；对于区间$[2^b, L_+]$内的长集合，则直接逐集合进行相似度验证。

（3）$L_+ \le 2^b - 1$，此时，所有可能与查询集合Q相似的集合均落在 LMB 的可索引范围内，查询过程可完全在 LMB 中执行。

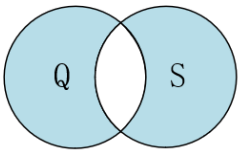
在第（1）种和第（2）种情况下，区间$[L_-, L_+]$内可能包含未被 LMB 索引的长集合，即长度大于$2^b - 1$的集合。对于这部分集合，本文不再为其构建索引，而是直接将其纳入候选集，直接与查询集合Q进行精确验证。考虑到实际数据集中较长集合通常只占较小比例，这种处理方式能够在保证正确性的同时简化查询流程。

3.4.2 基于长度过滤和对称差的键边界计算

长度过滤是确定键边界的有效工具，它定义了长度下界$L_- = \lceil |Q| * \delta \rceil$和长度上界$L_+ = \lfloor |Q| / \delta \rfloor$。因此，长度不在区间$[L_-, L_+]$内的集合不可能和查询集合$Q$相似，可被安全地过滤。

然而，长度过滤仅限定了与查询集合Q满足相似度阈值δ的候选集合的长度范围，所给出的约束相对宽松，仍可能产生较大的候选集。为了进一步限定这些候选集合和查询集合Q的不同元素的个数，本节引入对称差的概念。

定义 2（对称差） 集合S和查询集合Q的对称差为$(Q - S) \cup (S - Q)$，表示属于S而不属于Q的元素，以及属于Q而不属于S的元素所构成的集合。对称差的示意图如图 3.3 所示，其中，浅蓝色区域即为集合S和查询集合Q的对称差。



(Q-S)∪(S-Q)

图 3.3 对称差示意图

本章称$\lambda_{S,Q} = |(Q - S) \cup (S - Q)|$为集合$S$和查询集合$Q$的对称差的大小。若

集合S和查询集合Q相似，则它们之间的 Jaccard 相似度要大于等于相似度阈值δ，即$Jaccard(S,Q)\geq\delta$，可以进一步推导得到以下关系式：

$$Jaccard(S,Q)=\frac{|S\cap Q|}{|S\cup Q|}=\frac{|S\cap Q|}{|S|+|Q|-|S\cap Q|}\geq\delta$$

$$\Rightarrow|S\cap Q|\geq\frac{\delta}{1+\delta}(|S|+|Q|) \tag{3.1}$$

根据这一关系式，可以进一步推导得到定理 1。

定理 1：$\lambda_{S,Q}\leq\frac{1-\delta}{1+\delta}(|S|+|Q|)$

证明如下：

$$\lambda_{S,Q}=|(Q-S)\cup(S-Q)|=|S|+|Q|-2|S\cap Q|$$
$$\leq|S|+|Q|-\frac{2*\delta}{1+\delta}(|S|+|Q|)=\frac{1-\delta}{1+\delta}(|S|+|Q|)$$

因此，本章将$\Lambda_{S,Q}=\frac{1-\delta}{1+\delta}(|S|+|Q|)$定义为集合$S$和查询集合$Q$的对称差上界。对于任意与查询集合$Q$相似的集合$S$，其与查询集合$Q$之间的对称差均不超过对称差上界$\Lambda_{S,Q}$。本文发现，$\Lambda_{S,Q}$仅与$S$的长度有关，而与$S$中的具体元素无关，因此，对于特定的长度$L$，本章进一步定义$\Lambda_{L,Q}=\frac{1-\delta}{1+\delta}(L+|Q|)$为所有长度为$L$的集合与查询集合$Q$的对称差上界，称为查询集合$Q$长度特定$L$的对称差上界。基于该定义，本章只需针对特定的长度$L$计算对称差上界$\Lambda_{L,Q}$，而无需对每个集合$S\in D$分别计算对称差上界$\Lambda_{S,Q}$。因此，该方法能够有效减少计算开销，并提高查询效率。

根据定理 1，Q和任何相似于Q的集合之间的对称差不能超过$\Lambda_{L,Q}$。因此，可利用$\Lambda_{L,Q}$为不同长度$L\in[L_-,L_+]$构造更紧的键边界。具体而言，对于给定长度$L\in[L_-,L_+]$，本文将$\Lambda_{L,Q}$分配到Q的分区向量$Q.V$中，从而生成查询集合Q长度特定L的键边界。

本章采用第一分区优先分配策略（First Partition First Allocation，FPFA），其核心思想是尽可能地将$\Lambda_{L,Q}$分配到$Q.V$中靠前的分区（即靠前的分量值）。对于每个长度$L\in[L_-,L_+]$，尽可能从$Q.V$中靠前的分量值减去$\Lambda_{L,Q}$得到查询集合Q长度特定L的键下界$K_{L,-}$，称为 GetLBKey 算法。详细的 GetLBKey 算法在算法 1 中给出。

算法 1：GetLBKey

输入： Q：查询集合

$\Lambda_{L,Q}$：查询集合 Q 长度特定 L 的对称差上界

输出： $K_{L,-}$：查询集合 Q 长度特定 L 的键下界

1：$K_{L,-}.V \leftarrow Q.V$

2：$tmp \leftarrow \Lambda_{L,Q}$

3：for $i \leftarrow 1$ to M do

4：　if $K_{L,-}.v_i > tmp$ then

5：　　$K_{L,-}.v_i \leftarrow K_{L,-}.v_i - tmp$

6：　　break

7：　else

8：　　$tmp \leftarrow tmp - K_{L,-}.v_i$

9：　　$K_{L,-}.v_i \leftarrow 0$

10：return $K_{L,-}$

对于每个长度 $L \in [L_-, L_+]$，尽可能从 $Q.V$ 中靠前的分量值加上 $\Lambda_{L,Q}$ 得到查询集合 Q 长度特定 L 的键上界 $K_{L,+}$，称为 GetUBKey 算法。详细的 GetUBKey 算法在算法 2 中给出。

算法 2：GetUBKey

输入： Q：查询集合

$\Lambda_{L,Q}$：查询集合 Q 长度特定 L 的对称差上界

输出： $K_{L,+}$：查询集合 Q 长度特定 L 的键上界

1：$K_{L,+}.V \leftarrow Q.V$

2：$tmp \leftarrow \Lambda_{L,Q}$

3：for $i \leftarrow 1$ to M do

4：　if $K_{L,+}.v_i + tmp \le (2^b - 1)$ then

5：　　$K_{L,+}.v_i \leftarrow K_{L,+}.v_i + tmp$

6：　　break

7：　else

8：　　$tmp \leftarrow K_{L,+}.v_i + tmp - (2^b - 1)$

9：　　$K_{L,+}.v_i \leftarrow 2^b - 1$

10：return $K_{L,+}$

GetLBKey 首先初始化键下界 $K_{L,-}$（第 1 行）。如果从第一个分量值 $K_{L,-}.v_1$ 中减去 $\Lambda_{L,Q}$ 不会导致下溢，则直接从 $K_{L,-}.v_1$ 中减去 $\Lambda_{L,Q}$；否则，将下溢的部分分配给后续的分量值（第 2~9 行）。最后，返回键下界 $K_{L,-}$（第 10 行）。

GetUBKey 计算键上界 $K_{L,+}$ 的计算方式类似，保证不溢出的情况下，尽可能将 $\Lambda_{L,Q}$ 加到靠前的分量值中（第 1~9 行）。最后返回键上界 $K_{L,+}$（第 10 行）。

下面的定理 2 证明了 FPFA 的正确性。

定理 2：由 FPFA 生成的键下界 $K_{L,-}$ 和键上界 $K_{L,+}$ 不会导致假阴性。

证明：由于将 $\Lambda_{L,Q}$ 优先分配给 $Q.V$ 中靠前的分量值，易知，任何小于 $K_{L,-}$ 或者大于 $K_{L,+}$ 的键与查询集合 Q 之间的对称差都会超过 $\Lambda_{L,Q}$，这些键不可能与查询集合 Q 相似。因此，定理 2 成立。

在最坏的情况下，FPFA 将 $\Lambda_{L,Q}$ 分配到所有的 M 个分量值才完成键下界 $K_{L,-}$ 和键上界 $K_{L,+}$ 的计算，时间复杂度为 $O(M)$。考虑到实际中 M 通常较小，因此 FPFA 是高效的。

3.4.3 基于对称差的键过滤

在得到键下界 $K_{L,-}$ 和键上界 $K_{L,+}$ 之后，可以在 LMB 中查找到不小于键下界 $K_{L,-}$ 的键作为起始键，然后沿着叶节点的指针链进行遍历，将不大于键上界 $K_{L,+}$ 的键 K 都加入候选键集。然后，对于候选键集中的每个键 K，从其数组 $K.Vec$ 中取出生成该键 K 的所有集合 ID，并根据 ID 找到集合，验证这些集合与查询集合 Q 的 Jaccard 相似度。这种方法可能会效率低下，因为候选集合较多。为了提高效率，本节设计了一种有效的基于对称差的过滤技术来过滤不可能与查询集合 Q 相似的键。

当访问键下界 $K_{L,-}$ 和键上界 $K_{L,+}$ 之间的某个键 K 时，由定理 1 可知，若查询集合 Q 的分区向量 $Q.V$ 和键 K 的分区向量 $K.V$ 之间的绝对差的大于相似度阈值 δ，即 $\sum_{i=1}^{M}|Q.v_i - K.v_i| > \delta$，则键 K 与 Q 不可能相似，则可以过滤掉键 K，进而过滤掉生成键 K 的所有集合。

一般情况下，需要遍历 M 个分量值才可以过滤掉键 K。为了进一步提高过滤效率，本节提出了一种提前中止策略，可以提前结束过滤过程。具体而言，在计算绝对差的过程中，若当前访问第 $j(1 \le j \le M)$ 个分量值，前 j 个累积的绝对差

已经大于相似度阈值δ，即$\sum_{i=1}^{j}\left|Q.v_i - K.v_i\right| > \delta$，则键$K$不可能和$Q$相似，可以安全地过滤掉键$K$，就可以提前中止计算。

3.4.4 验证

在验证集合S和Q的相似度时，本章使用传统的方法，即通过计算两集合的实际覆盖并按 Jaccard 相似度公式验证。

基于以上描述，在算法 3 中给出 LMB-SD 算法。LMB-SD 首先将结果集RS初始化（第 1 行），然后计算下界L_-和长度上界L_+（第 2 行）。针对前述三种长度情况分别进行处理，并计算需在 LMB 中执行查询的最大长度$\max L$（第 3~10 行）。对于查询Q在 LMB 中执行的部分，循环对每个长度$L \in [L_-, \max L]$进行如下处理：首先检查B_L是否为空，如果为空，则继续下一个长度（第 12~13 行）；否则，计算对称差上界$\Lambda_{L,Q}$，并通过 GetLBKey 和 GetUBKey 算法分别计算键下界$K_{L,-}$和键上界$K_{L,+}$（第 15~16 行）。随后，应用键过滤策略生成候选集，并验证候选集来获得最终的结果（第 17~25 行）。最后，返回结果集（第 26 行）。

示例 3.4：图 3.4 展示了 LMB-SD 算法的一个查询示例，给定一个查询集合$Q=\{e_1,e_2,e_6,e_7,e_{11},e_{12},e_{16},e_{20}\}$和相似度阈值$\delta=0.7$。通过键生成策略可以得到$Q.K=[2,2,2,2]$。根据长度过滤可以得到长度下界$L_-=6$和长度上界$L_+=11$。集合$S_1$和$S_{10}$不满足长度要求而被排除。通过计算得到$\Lambda_{6,Q}=\Lambda_{7,Q}=2$（$\Lambda_{8,Q}=2$，但是长度为 8 的子树为空，因此不会计算$\Lambda_{8,Q}$），和$\Lambda_{9,Q}=\Lambda_{10,Q}=\Lambda_{11,Q}=3$，采用 GetLBKey 和 GetUBKey 算法得到$K_{6,-}=K_{7,-}=[0,2,2,2]$、$K_{6,+}=K_{7,+}=[4,2,2,2]$、$K_{9,-}=K_{10,-}=K_{11,-}=[0,1,2,2]$和$K_{9,+}=K_{10,+}=K_{11,+}=[5,2,2,2]$。由于$S_2.K$=[0,2,1,3]小于$K_{6,-}$，$S_3.K=[5,0,1,0]$大于$K_{6,+}$，$S_2$和$S_3$不在键边界内而被安全地过滤。然后，应用基于对称差的键过滤策略之后，$\{S_7,S_8,S_9\}$被过滤了，$\{S_4,S_5,S_6\}$（具有绿色背景）成为最终的候选集合。经过 Jaccard 验证，得到结果集为S_5和S_6。

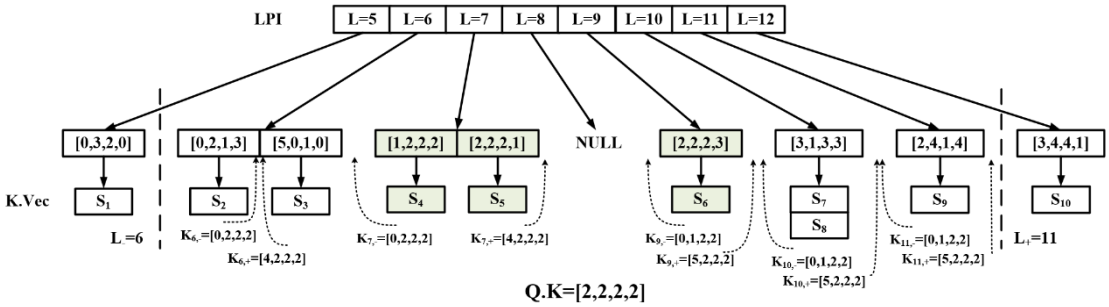


图 3.4 LMB-SD 示意图

算法 3：LMB-SD

输入： Q：查询集合

δ：相似度阈值

输出： RS：数据集 D 中与查询集合 Q 满足相似度阈值 δ 的所有集合

1：$RS \leftarrow NULL$

2：$L_- \leftarrow \lceil |Q| * \delta \rceil, L_+ \leftarrow \lfloor |Q| / \delta \rfloor$

3：if $L_- > (2^b - 1)$ then

4：　append each $S \in D$ with $|S| \in [L_-, L_+]$ into RS if $Jaccard(S,Q)$ is $True$

5：　return RS

6：if $L_+ > (2^b - 1)$ then

7：　append each $S \in D$ with $|S| \in [2^b, L_+]$ into RS if $Jaccard(S,Q)$ is $True$

8：　$\max L \leftarrow 2^b - 1$

9：else

10：　$\max L \leftarrow L_+$

11：foreach L in $[L_-, \max L]$ do

12：　if $LPI[L]$ is $NULL$ then

13：　　continue

14：　$B_L \leftarrow LPI[L]$

15：　$\Lambda_{L,Q} \leftarrow \frac{1-\delta}{1+\delta}(L + |Q|)$

16：　$K_{L,-} \leftarrow GetLBKey(Q, \Lambda_{L,Q}), K_{L,+} \leftarrow GetUBKey(Q, \Lambda_{L,Q})$

17：　foreach $K \in [K_{L,-}, K_{L,+}]$ indexed in B_L do

18：　　$sum \leftarrow 0, flag \leftarrow True$

19：　　for $i \leftarrow 1$ to M do

20：　　　$sum \leftarrow sum + |Q.v_i - K.v_i|$

21：　　　if $sum > \Lambda_{L,Q}$ then

22：　　　　$flag \leftarrow False$

23：　　　　break

24：　　if $flag$ is $True$ then

25：　　　append each $S \in K.Vec$ into RS if $Jaccard(S,Q)$ is $True$

26：return RS

3.5 复杂度分析

3.5.1 空间复杂度

LMB 索引：假设数据集 D 总共生成了 N 个键，且数据集被均匀划分为 P 个分区，则每个分区包含 N/P 个键。每个 B_L 的空间开销为：

$$\frac{N}{P(f-1)}((f-1)X+Y)+\frac{N-P}{P(f-1)}((f-1)X+fY)$$

$$=(\frac{2N}{P}-1)X+\frac{f(\frac{N}{P}-1)+\frac{N}{P}}{f-1}Y$$

$$=\frac{2N-P}{P}X+\frac{f(N-P)+N}{P(f-1)}Y$$

其中，X 表示存储一个键的空间开销，Y 表示存储一个指针的空间开销。详细分析如下：

（1）每个 B_L 约有 $\frac{N}{P(f-1)}$ 个叶节点，每个叶节点存储 $f-1$ 个键和一个指向下一个叶节点的指针，所以其空间开销为 $\frac{N}{P(f-1)}((f-1)X+Y)$；

（2）每个 B_L 的高度为 $\log_f \frac{N}{P}$，因此，约有 $\sum_{i=0}^{\log_f \frac{N}{P}-1} f^i \approx \frac{\frac{N}{P}-1}{f-1}=\frac{N-P}{P(f-1)}$ 个内部节点，每个内部节点存储 $f-1$ 个键和 f 个指针，因此其空间开销为 $\frac{N-P}{P(f-1)}((f-1)X+fY)$。

整个集合数据集上有 P 个分区，因此，所有 B_L 的空间开销为 $P*(\frac{2N-P}{P}X+\frac{f(N-P)+N}{P(f-1)}Y)=(2N-P)X+\frac{f(N-P)+N}{f-1}Y$。最后，还需要加上 *K.Vec* 的开销 $|D|*Z$ 和 LPI 的开销 $(2^b-1)*Y$，Z 是存储一个集合 *ID* 的空间开销。因此，LMB 的总空间开销为：$(2N-P)X+\frac{f(N-P)+N}{f-1}Y+|D|*Z+(2^b-1)*Y$。

LMB-SD 算法：通过分析算法 3 给出了 LMB-SD 算法的代码，可以发现，LMB-SD 算法只使用了常数级别的空间，因此空间复杂度为 $O(1)$。

3.5.2 时间复杂度

假设长度上界 $L_+ \le (2^b - 1)$，即查询都在 LMB 索引中执行，设 $A = L_+ - L_-$，执行 A 次 FPFA 策略生成键边界的时间开销 $O(AM)$。在 LMB 中查找键下界 $K_{L,-}$ 的时间复杂度为 $O(A\log_f(N/P))$。键过滤的时间成本为 $O(N_1 * M)$，其中，N_1 是位于键边界 $[K_{L,-}, K_{L,+}]$ 内的键数量，验证的时间开销为 $O(N_2 * V)$，其中，N_2 是候选集合的数量，V 是验证两个集合的平均成本。因此，LMB-SD 的时间复杂度为 $O(AM + A\log_f(N/P) + N_1 * M + N_2 * V)$。

3.6 实验分析

本章中所有的实验均在一台配备 13th Gen Intel(R) Core(TM) i5-1340P CPU@1.9GHz 处理器及 32GB 内存的个人计算机上进行。实验环境基于 Windows 11 操作系统，采用 C++编程语言实现，代码使用 GCC 进行编译，并在 Microsoft Visual Studio Professional 2022 专业版环境中构建。

3.6.1 数据集

表 3.4 数据集的统计信息

数据集	基数	元素数量	最大长度	最小长度	平均长度
KOSARAK	990002	41270	2498	1	8.1
RETAIL	81735	16103	76	1	10.3
LASTFM	960585	992	984	1	4.12
TWEETS	20000000	1847534	38	1	5.48

本章使用以下四个真实数据集进行实验：

（1）KOSARAK（http://fimi.uantwerpen.be/data/）：包含来自一个匈牙利在线新闻门户的 990002 个集合，每个集合代表用户会话中的一个点击流序列。

（2）RETAIL（https://recsyswiki.com/wiki/Grocery_shopping_datasets）：包含 81735 个集合，每个集合代表来自比利时一家匿名零售商店的客户交易。

（3）LASTFM（http://www.dtic.upf.edu/ocelma/Music/RecommendationDataset/lastfm-1K.html）：包含 960585 个集合，每个集合代表特定用户的歌曲收听序列。

（4）TWEETS[68]：包含 20000000 个集合，每个集合代表一条推文的内容。

这四个数据集的详细统计信息在表 3.4 中。对于每个数据集，本节随机选择 1000 个集合作为查询集合，统计平均每个查询的执行时间。

3.6.2 对比索引技术

本章将 LMB 与 MBQ[47]、DualTrans[7]、LES3[51]和 LeBQ+[6]这四个算法中使用的索引进行比较，主要关注于索引构建时间、索引大小和索引高度。对于基于 B+树的算法，扇出 f 选择 16。表 3.5 展示了索引构建时间的统计结果。

表 3.5 索引构建时间（s）

	KOSARAK	RETAIL	LASTFM	TWEETS
MBQ	1.435	0.137	2.437	33.305
DualTrans	104.440	7.539	94.234	2896.46
LES3	12120.285	1384.398	9601.573	58160.268
LeBQ+	0.681	0.089	0.407	21.318
LMB	0.905	0.191	0.326	22.321

如表 3.5 所示，由于基于深度学习的训练策略，LES3 具有最长的索引构建时间。MBQ 的构建效率低于 LMB，主要是因为它依赖于 k-means 聚类算法。LeBQ+和 LMB 的构建时间相似，LMB 在 LASTFM 上更快。这种改进归功于 LMB 基于长度的分区策略，它减少了每个分区的集合数量，简化了索引构建过程。表 3.6 展示了索引大小的统计结果。

表 3.6 索引大小（MB）

	KOSARAK	RETAIL	LASTFM	TWEETS
MBQ	10.514	1.542	9.227	159.097
DualTrans	127.781	10.618	123.271	2569.65
LES3	4.542	1.217	1.253	38.3
LeBQ+	10.278	1.465	9.064	157.843
LMB	10.394	1.469	9.088	157.846

如表 3.6 所示，LMB 的空间开销低于 MBQ，略高于 LeBQ+。虽然 LMB 构建了多棵子树并且增加了 LPI 的空间开销，但是其构建的 B+树高度较低，而且内部节点更少，因此索引空间开销较小。相对于 MBQ 中的 k-means 聚类分区，LMB 中使用的基于集合长度的分区更简单且确定性更高，分区效率更高且很好地支持数据的更新。尽管 LES3 实现了最小的空间开销，它对机器学习的依赖使其不太适合数据更新频繁的环境。

LeBQ+在整个数据集构建唯一全局 B+树索引，而 LMB 在每个分区上都构建

了 B^+树索引，易知，相对于 $LeBQ^+$，LMB 具有更低的索引高度。表 3.7 进一步对比了 LMB 和 $LeBQ^+$的索引高度，LMB-Min 代表 LMB 索引中 B^+子树的最小高度，LMB-Max 代表 LMB 中 B^+子树的最大高度，LMB-Avg 代表 LMB 中所有 B^+子树的平均高度。可以看到，相比于 $LeBQ^+$，LMB 索引的高度显著降低，进而提升查询效率。

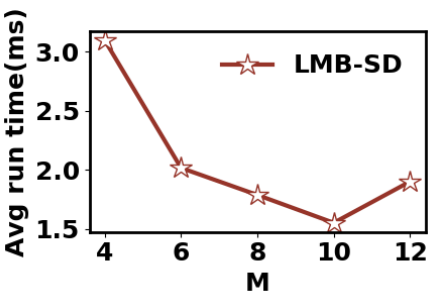
表 3.7 索引高度

	KOSARAK	RETAIL	LASTFM	TWEETS
$LeBQ^+$	6	5	6	6
LMB-Avg	3.13	1.39	2.78	0.93
LMB-Min	1	1	1	1
LMB-Max	4	4	4	5

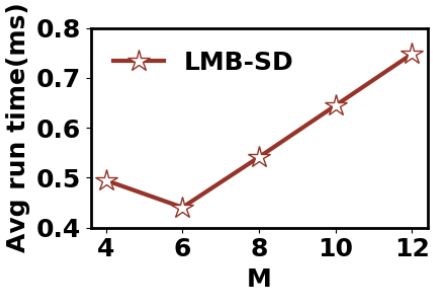
3.6.3 分区数对查询效率的影响

为了评估分区数 M 对查询效率的影响，本章固定相似度阈值 δ=0.7，并将分区数 M 从 4 变化到 12。实验结果如图 3.5 所示。

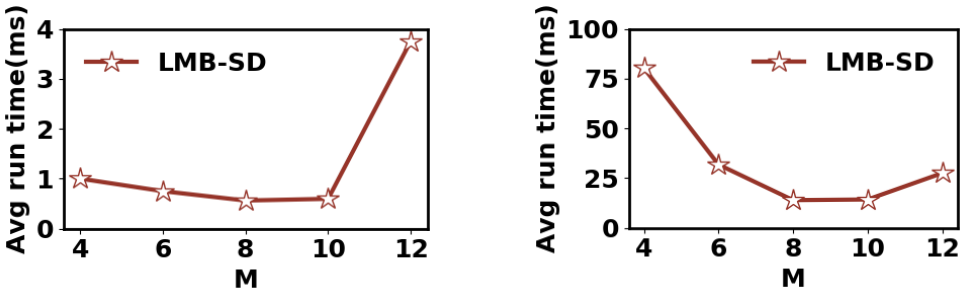
从图 3.5 中可以观察到，在所有的四个数据集中，随着分区数 M 的增加，LMB-SD 的平均查询时间最初减少，然后增加。这是因为随着分区数 M 的增加，不同元素映射到同一分区的概率降低，从而提高了集合区分度。然而，当分区数 M 进一步增加时，b 降低，从而减少了 LMB 索引最大支持集合长度并增加了需要直接验证的集合数量。考虑到在四个数据集上，LMB-SD 的性能在 M =8 时接近最佳，本章在后续实验中采用此作为默认的分区数。



（a）KOSARK



（b）RETAIL

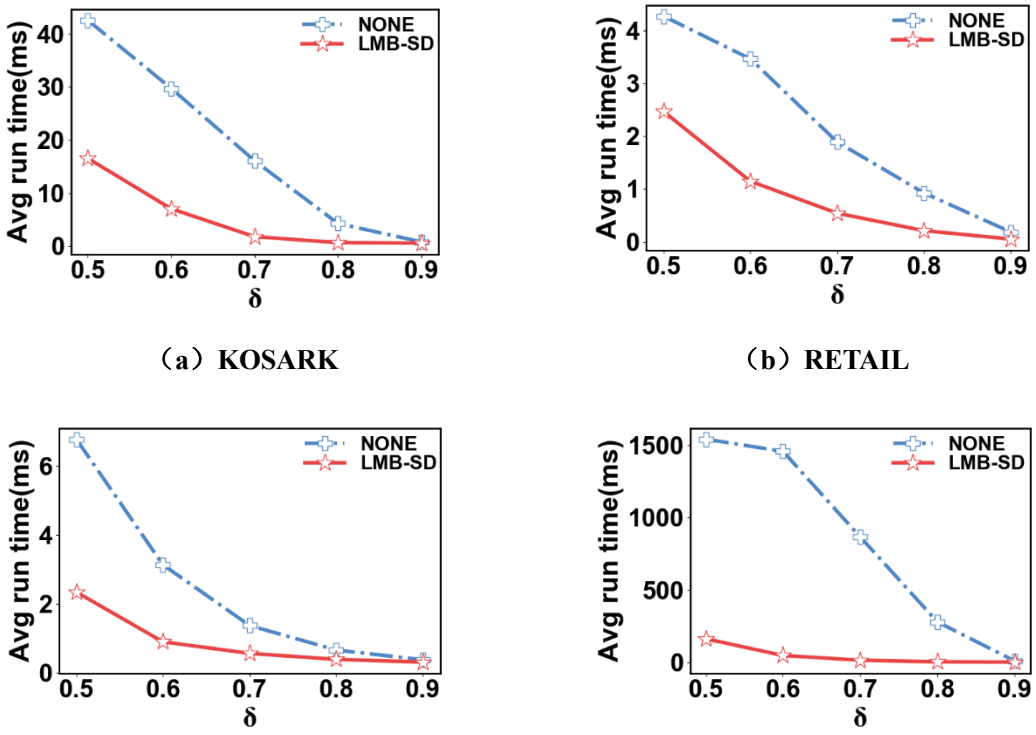


（c）LASTFM　　（d）TWEETS

图 3.5 分区数对查询效率的影响

3.6.4 过滤策略的效率

为了评估过滤策略的效率，本节在 LMB-SD 算法中删除了基于对称差的键过滤策略，记为 NONE。平均查询时间的对比结果如图 3.6 所示。



（a）KOSARK　　（b）RETAIL

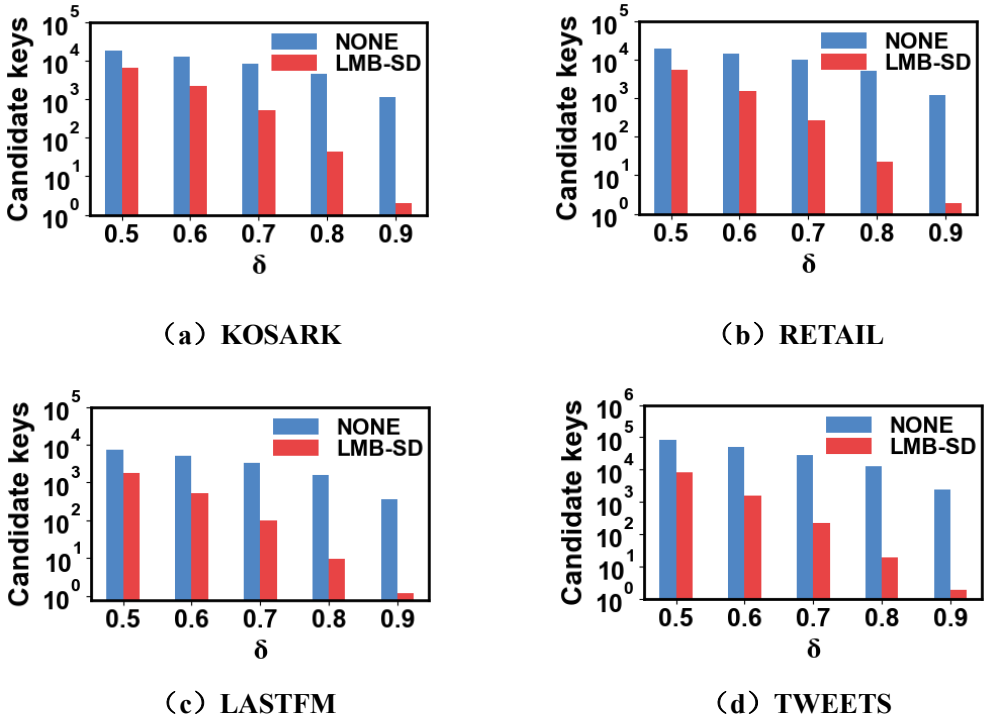
（c）LASTFM　　（d）TWEETS

图 3.6 过滤策略的效率

从图 3.6 中可以看出，这两个算法的平均查询时间随着相似度阈值 δ 的增加而减少。这是因为随着相似度阈值 δ 的增加，长度下界 L_- 变大，长度上界 L_+ 变小，减少了候选集合。因为基于对称差的键过滤策略，LMB-SD 算法始终由于 NONE 算法。具体而言，在 KOSARAK，RETAIL，LASTFM 和 TWEETS 上，

LMB-SD 实现的效率提升高达 88.9%，77.2%，71.3%和 98.9%。

为了进一步解释基于对称差的键过滤策略的优秀性能，图 3.7 对比了 LMB-SD 和 NONE 的候选键数量（这里是指经过键过滤之后得到的候选键）。从图 3.7 中可以看出，基于对称差的键过滤策略可以显著减少了候选键的数量，其有效性随着相似度阈值δ的增加而增加。



（a）KOSARK　　（b）RETAIL

（c）LASTFM　　（d）TWEETS

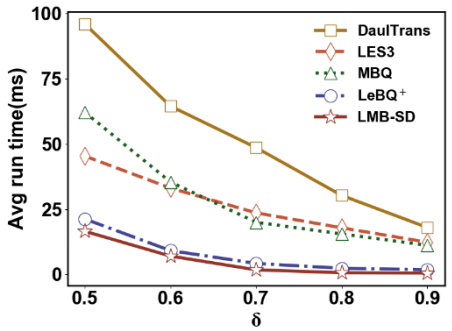
图 3.7 过滤策略的候选键数量

3.6.5 对比其他算法

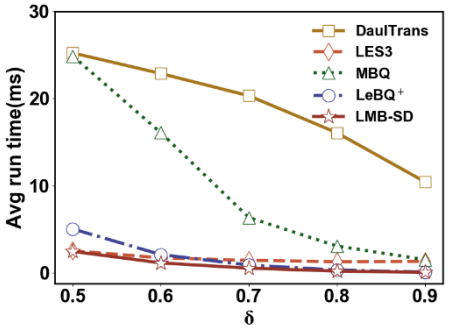
本文研究的是精确集合相似度查询，因此，本文的算法查询结果的准确性是需要保证的。为了评估 LMB-SD 的查询效率，将它与 MBQ，DualTrans，LES3 和 LeBQ^{+}这四种精确集合相似度查询算法进行了比较。实验结果如图 3.8 所示。

从图 3.8 中可以看出，LMB-SD 优于其他算法，这表明其有效性。LMB-SD 的优势随着相似度阈值δ的增加而增加，这是因为随着相似度阈值δ的增加，长度过滤的剪枝能力增加。与 MBQ 相比，LMB-SD 可以直接确定满足相似度阈值δ的键边界，避免了逐节点调用 Upper 函数带来的高昂开销。与 LeBQ^{+}相比，LMB-SD 通过采用基于集合长度的分区策略，显著降低了索引高度，进而提高了查询速度。当δ=0.9 时，MBQ 在四个数据集上的平均查询时间分别为 11.18ms、1.52ms、6.97ms 和 23.61ms，LeBQ^{+}的平均查询时间分别为 1.82ms、0.09ms、0.56ms 和 1.38ms，而 LMB-SD 的平均查询时间分别为 0.58ms、0.05ms、0.31ms 和 0.95ms，

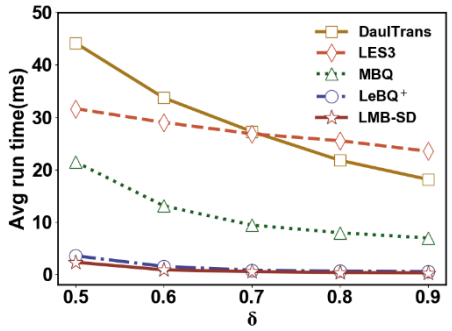
LMB-SD 的查询效率比 MBQ 提高了最高 29.40 倍，比 LeBQ⁺提高了最高 2.14 倍。在 TWEETS 上的性能表明了 LMB-SD 适用于大型数据集。



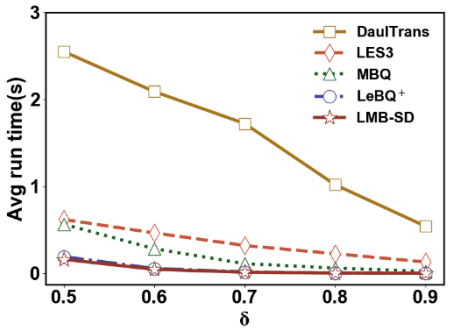
（a）KOSARK



（b）RETAIL



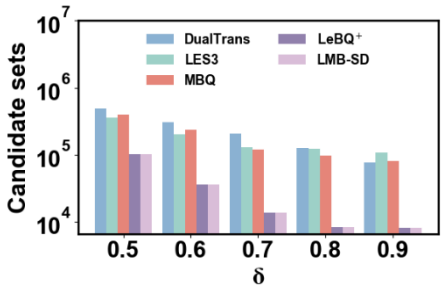
（c）LASTFM



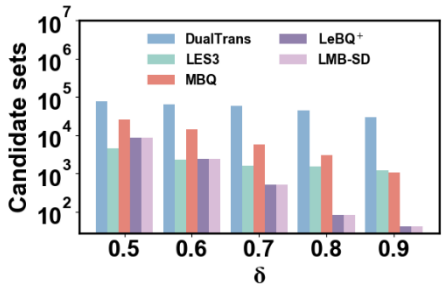
（d）TWEETS

图 3.8 不同算法的平均查询时间

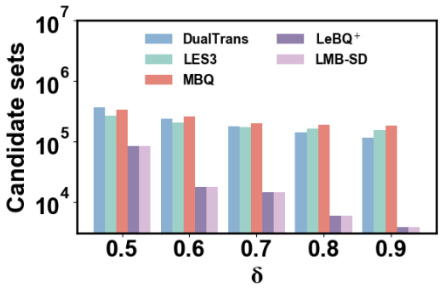
LMB-SD 的优秀性能很大程度上在于它生成了较少的候选集，图 3.9 展示了不同算法的候选集数量。当相似度阈值 δ=0.9 时，在四个数据集中，LMB-SD 的候选集数量分别比 MBQ 和 LES3 减少了 29 倍和 186 倍。之所以生成较少的候选集，其中一个关键原因在于，LMB-SD 基于键而不是基于集合生成候选集。如果过滤键 K，则同时过滤 $K.Vec$ 中的所有集合，从而生成更少的候选集，并且 LMB-SD 具有更高的候选集生成效率。



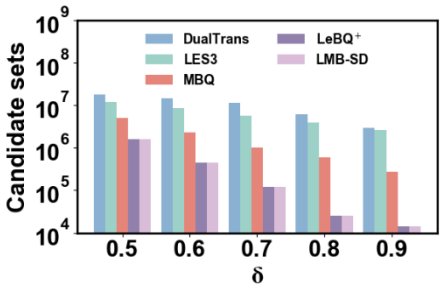
（a）KOSARK



（b）RETAIL



（c）LASTFM



（d）TWEETS

图 3.9 不同算法的候选集数量

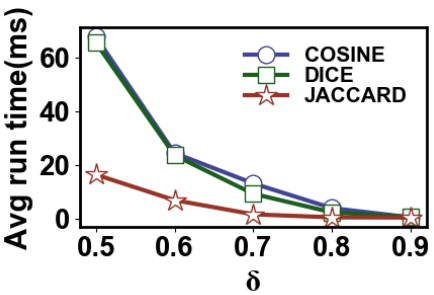
3.6.6 扩展到其他相似度函数

除了 Jaccard 相似度，LMB-SD 算法还支持扩展到 Cosine、Dice 等其他相似度函数。扩展到这些函数的关键之处在于计算对应的长度下界 L_-、长度上界 L_+ 以及查询集合 Q 长度特定 L 的对称差上界 $\Lambda_{L,Q}$，如表 3.8 所示。

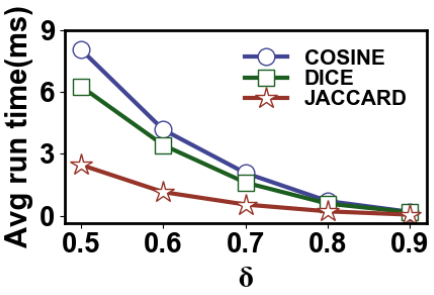
表 3.8 LMB-SD 扩展到其他相似度函数

相似度函数	L_-	L_+	$\Lambda_{L,Q}$
Jaccard	$\lceil \lvert Q\rvert * \delta \rceil$	$\lfloor \frac{\lvert Q\rvert}{\delta} \rfloor$	$\lfloor \frac{1-\delta}{1+\delta}(L+\lvert Q\rvert) \rfloor$
Cosine	$\lceil \lvert Q\rvert * \delta^2 \rceil$	$\lfloor \frac{\lvert Q\rvert}{\delta^2} \rfloor$	$\lfloor L+\lvert Q\rvert - 2\delta\sqrt{L\lvert Q\rvert} \rfloor$
Dice	$\lceil \lvert Q\rvert * \frac{\delta}{2-\delta} \rceil$	$\lfloor \frac{2-\delta}{\delta}\lvert Q\rvert \rfloor$	$\lfloor (1-\delta)(L+\lvert Q\rvert) \rfloor$

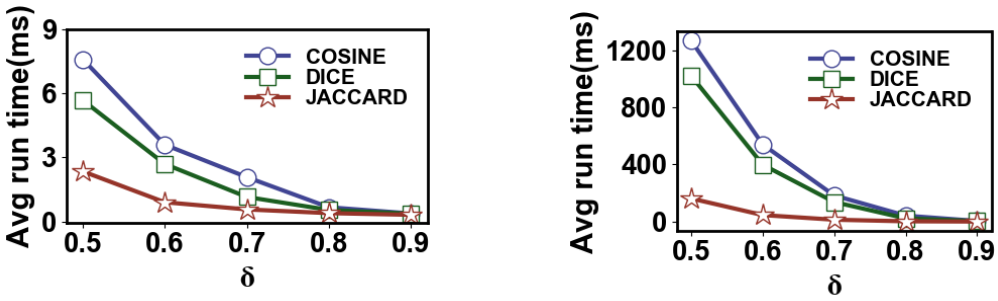
为了评估 LMB-SD 算法在不同相似度函数下的性能，本节使用 Jaccard、Cosine 和 Dice 函数分别进行实验。实验对比结果如图 3.10 所示。在所有数据集中，随着相似度阈值 δ 的增加，每个相似度函数生成的候选集都会减少，从而减少运行时间。其中，Jaccard 生成的候选集最少，运行时间最短。



（a）KOSARK



（b）RETAIL



（c）LASTFM　　（d）TWEETS

图 3.10 不同相似度函数下的平均查询时间

3.7 本章小结

针对现有集合相似度查询方法在集合长度信息利用不足、逐节点相似度上界计算开销较高以及单一索引结构在大规模数据集上效率受限等问题，本章提出了基于长度分区的多 B^+树索引结构 LMB 及基于对称差的集合相似度查询算法 LMB-SD。首先，本文通过长度分区将不同长度的集合划分到独立分区中，并在各分区内分别构建 B^+树子索引，从而形成多 B^+树索引结构 LMB。该结构能够有效缩小搜索空间，降低索引高度，并提高大规模数据集上的索引组织效率。其次，在 LMB 索引基础上，本文提出了 LMB-SD 算法。该算法利用长度过滤进行初步剪枝，并通过基于长度过滤和对称差的键边界计算策略快速生成长度特定的键边界，避免了传统方法中逐节点计算相似度上界的复杂过程；进一步结合基于对称差的键过滤策略，能够有效减少候选集规模，提高查询效率。最后，通过在多个真实数据集上的实验分析，验证了 LMB 索引和 LMB-SD 算法的有效性。

从计算机系统结构的角度分析，LMB 通过长度分区将全局索引分解为多个局部 B^+子树，有效降低了索引高度与节点访问次数；LMB-SD 通过基于对称差的键边界计算策略，避免了传统方法中逐节点计算相似度上界的高昂开销。这两项优化分别从存储层次与查询执行路径两个层面提升了系统整体效率，为下一章进一步研究基于相异差的集合相似度查询算法奠定了基础。

4 基于长度分区和相异差的集合相似度查询算法

4.1 引言

尽管 LMB-SD 通过基于长度过滤和对称差的键边界计算策略避免了传统方法中逐节点计算相似度上界的复杂过程，并取得了较好的查询性能，但其生成的键边界仍相对宽泛。在部分查询场景下，较宽的键边界会引入较多无效索引扫描，导致候选键集的规模偏大，从而增加后续键过滤与验证阶段的计算开销，限制了算法性能的进一步提升。

造成这一问题的根本原因在于，LMB-SD 采用对称差对查询集合与候选集合之间的差异进行整体刻画。虽然这种方式能够有效约束键边界范围，但对称差本质上将两个方向的差异统一表示为一个整体量，未能充分区分“查询集合相对于候选集合的差异”和“候选集合相对于查询集合的差异”。在键边界计算过程中，这种整体性描述会导致边界约束不够紧致，从而影响候选控制能力。

基于上述分析，本章进一步引入相异差的思想，将原有的整体差异拆分为两个相异的单向差，并在此基础上提出基于相异差的集合相似度查询算法 LMB-AD。LMB-AD 包括以下三个核心部分：

（1）基于长度过滤和相异差的键边界计算：通过分别利用两个单向差对键下界和键上界进行约束，生成更加紧凑的键边界，从而进一步减少无效索引扫描和候选键集规模。

（2）基于相异差的键过滤策略：在新的边界计算机制基础上，进一步设计与之匹配的键过滤策略，用于排除位于键边界范围内但与查询集合不相似的集合，从而提高候选筛选精度。

（3）验证阶段优化：通过引入基于最小覆盖和提前中止的验证策略，加速候选集验证过程，进一步降低整体查询开销。

因此，LMB-AD 在保持 LMB 索引结构不变的基础上，通过更细粒度的差异建模和更紧致的搜索边界控制，进一步提升了集合相似度查询的整体效率。

本章后续内容安排如下：4.2 节给出算法总体框架；4.3 节介绍索引构建；4.4 节介绍 LMB-AD 算法的关键组成，包括基于长度过滤和相异差的键边界计算、基于相异差的键过滤以及基于最小覆盖和提前中止的验证策略；4.5 节分析算法的空间复杂度和时间复杂度；4.6 节通过实验分析验证 LMB-AD 算法的有效性和

优越性。

4.2 算法总体框架

本章提出的 LMB-AD 算法同样遵循索引构建与查询执行两阶段架构，如图 4.1 所示。与 LMB-SD 相比，LMB-AD 采用相同的索引结构（LMB），但其核心改进在于查询执行阶段引入了基于相异差的键边界计算，从而生成更加紧凑的搜索边界。

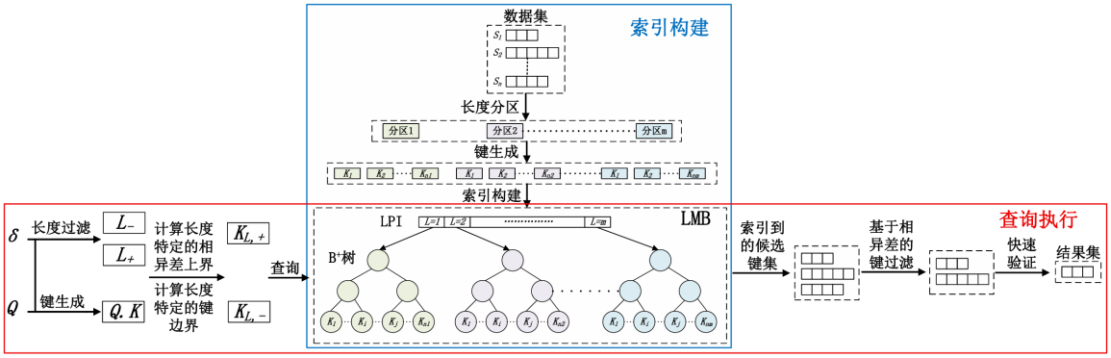


图 4.1 LMB-AD 算法总体框架

索引构建阶段：如图 4.1 中的蓝色边框区域所示，与第三章 LMB-SD 相同（详见 3.3 节），本章不再赘述。

查询执行阶段：如图 4.1 中的红色边框区域所示，对于给定的查询集合 Q 和相似度阈值 δ，首先计算 Q 的一维整型键 $Q.K$（键生成），并计算其长度下界 L_- 与长度上界 L_+，筛选出满足相似度阈值的候选长度分区（长度过滤）。区别于 LMB-SD 采用对称差进行整体约束，LMB-AD 将对称差拆分为两个相异的单向差（上界差与下界差），并基于长度过滤和相异差设计键边界计算策略，生成更加紧凑的键下界与键上界。在键边界范围内，算法依次访问各 B⁺子树中的键，采用基于相异差的键过滤策略对键进行深度筛选，排除不可能相似的键及其对应的所有集合，生成候选集。最后，对候选集中的集合执行基于最小覆盖和提前中止的精确验证，得到最终结果集。

上述框架通过长度分区实现了搜索空间的初步缩减，利用基于相异差的键边界计算策略生成了比 LMB-SD 更紧凑的搜索边界，并结合基于相异差的键过滤与优化的验证策略，进一步减少了候选键集规模与验证开销，从而提升了整体查询效率。

4.3 索引构建

本章算法同样基于 LMB 索引实现。LMB 索引的构建方法、键生成策略及长度分区索引 LPI 已在 3.2 节中详细阐述，此处不再重复。

4.4 查询执行

4.4.1 基于长度过滤和相异差的键边界计算

为了克服对称差的缺点，本章引入了一种有效的相异差，它将对称差分解为两个单向差：上界差$\eta_{S,Q}=|Q-S|$，它代表在Q中但不在S中的元素数量，如图 4.2 中的（a）图所示；下界差$\mu_{S,Q}=|S-Q|$，它代表在S中而不在Q中的元素数量，如图 4.2 中的（b）图所示。



（a）上界差示意图 **（b）下界差示意图**

图 4.2 上界差和下界差的示意图

基于此，本节提出定理 3 和定理 4。

定理 3：如果S和Q相似，那么$\eta_{S,Q}\le\dfrac{|Q|-|S|*\delta}{1+\delta}$

证明如下：

$$\eta_{S,Q}=|Q-S|=|Q|-|Q\cap S|$$

$$\le|Q|-\frac{\delta}{1+\delta}(|Q|+|S|)=\frac{|Q|-|S|*\delta}{1+\delta}$$

定理 4：如果S和Q相似，那么$\mu_{S,Q}\le\dfrac{|S|-|Q|*\delta}{1+\delta}$

证明如下：

$$\mu_{S,Q}=|S-Q|=|S|-|Q\cap S|$$

$$\le|S|-\frac{\delta}{1+\delta}(|Q|+|S|)=\frac{|S|-|Q|*\delta}{1+\delta}$$

本章将$\nabla_{S,Q}=\frac{|Q|-|S|*\delta}{1+\delta}$和$\Delta_{S,Q}=\frac{|S|-|Q|*\delta}{1+\delta}$分别作为上界差$\eta_{S,Q}$和下界差$\mu_{S,Q}$的上界。这两个上界仅取决于集合$S$的长度，而与其具体的元素无关，因此，对于长度为$L$的集合，本节定义$\nabla_{L,Q}=\frac{|Q|-L*\delta}{1+\delta}$为查询集合$Q$长度特定$L$的上界差的上界，$\Delta_{L,Q}=\frac{L-|Q|*\delta}{1+\delta}$为查询集合$Q$长度特定$L$的下界差的上界。

本节同样采用 FPFA 策略来计算长度特定的键边界。由于最多有$\nabla_{L,Q}$元素在Q中而不在S中，从$Q.V$中减去$\nabla_{L,Q}$来计算键下界$K_{L,-}$，具体操作为：如果第一个分量值$Q.v_1$减去$\nabla_{L,Q}$没有下溢，则直接$Q.v_1-\nabla_{L,Q}$，否则，从后面的分量值中减去溢出值，得到最小的键。类似地，由于最多有$\Delta_{L,Q}$元素在S中而不在Q中，尽可能地将$\Delta_{L,Q}$添加到$Q.V$中靠前的分量值得到键上界$K_{L,+}$。这两个算法和 GetLBKey 和 GetUBKey 大致相同，需要将输入的对称差上界$\Lambda_{L,Q}$改成了$\nabla_{L,Q}$或者$\Delta_{L,Q}$。

示例 4.1：给定$Q.K=[2,2,2,2]$和$\delta=0.7$，计算长度$L=6$的键边界，可以得到对称差上界$\Lambda_{6,Q}=2$。通过从$Q.K$的靠前的分量值中减去 2 和增加 2，可以得到键下界$K_{6,-}=[0,2,2,2]$和键上界$K_{6,+}=[4,2,2,2]$。采用本章中提出的基于相异差的键边界计算策略，可以得到$\nabla_{6,Q}=2$和$\Delta_{6,Q}=0$。通过 FBFA 策略，可以得到键下界$K_{6,-}=[0,2,2,2]$和键上界$K_{6,+}=[2,2,2,2]$。对比可知，虽然两种方式生成的键下界相同，但基于相异差的键边界计算策略生成了更小的键上界，因此，它生成了更紧凑的键边界，比基于对称差的键边界计算策略生成的键边界缩小了近一半。

4.4.2 基于相异差的键过滤

本节设计了一种基于相异差的键过滤策略，进一步过滤掉不可能与查询集合Q相似的键。直观地说，如果键K和Q满足以下条件之一，则生成键K的所有集合都不可能和Q相似：

条件 1：$Q.v_i-K.v_i>\nabla_{L,Q},\forall i\in[1,M]\&Q.v_i>K.v_i$

条件 2：$K.v_i-Q.v_i>\Delta_{L,Q},\forall i\in[1,M]\&Q.v_i<K.v_i$

条件 3：$\sum_{i=1}^{M}\max\{Q.v_i-K.v_i,0\}>\nabla_{L,Q}$

条件 4：$\sum_{i=1}^{M}\max\{K.v_i-Q.v_i,0\}>\Delta_{L,Q}$

本节只需要验证条件 3 和条件 4。因为条件 1 成立，则条件 3 必成立，但是条件 3 成立，条件 1 不一定成立；同理，条件 2 成立，条件 4 必成立，而条件 4 成立，条件 2 不一定成立。

为了减轻计算条件 3 和条件 4 的开销，本节引入了提前中止策略，它与 3.3.3 节中提出的提前中止策略大同小异。假设当前访问到第 $j(1 \le j \le M)$ 个分量值，如果 $\sum_{i=1}^{j} \max\{Q.v_i - K.v_i, 0\} > \nabla_{L,Q}$ 或者 $\sum_{i=1}^{j} \max\{K.v_i - Q.v_i, 0\} > \Delta_{L,Q}$，提前中止过滤过程，这显著提高了过滤效率。

4.4.3 基于最小覆盖和提前中止的验证

在验证集合 S 和 Q 的相似度时，传统的方法通过计算两集合的实际覆盖并按 Jaccard 相似度公式验证。然而，由于计算实际覆盖的复杂度较高，本节引入了提前中止策略。该策略基于以下观察：若 S 和 Q 相似，则它们的最小覆盖满足条件 $MinO_{S,Q} \ge \frac{1-\delta}{1+\delta}(|S|+|Q|)$。本章假定集合 S 和 Q 是值序有序，在验证时，对 S 的第 i 个位置和 Q 的第 j 个位置，若 $O(i,j) + \min(|S_U|, |Q_U|) < MinO_{S,Q}$，则验证可提前终止。其中，$O(i,j)$ 表示当前已探测到的覆盖值，$\min(|S_U|, |Q_U|)$ 表示 S 和 Q 未探测部分的最小长度。同样的，若当前已探测到的覆盖值大于最小覆盖，即 $O(i,j) \ge MinO_{S,Q}$，那么集合 S 和 Q 相似，可以中止验证过程。通过这种提前中止策略，可显著加快验证的过程，在本章中该验证策略记为 $J(S,Q)$。

基于以上描述，在算法 4 中给出 LMB-AD 算法。

LMB-AD 前面的查询操作和 LMB-SD 的大致相同，只有第 4 行和第 7 行使用了基于最小覆盖和提前中止的验证策略（第 1~10 行）。对于查询 Q 在 LMB 中执行的部分，循环对每个长度 $L \in [L_-, \max L]$ 进行如下处理：首先检查 B_L 是否为空，如果为空，则继续下一个长度（第 12~13 行）；否则，计算相异差上界 $\nabla_{L,Q}$ 和 $\Delta_{L,Q}$，并计算键下界 $K_{L,-}$ 和键上界 $K_{L,+}$（第 15~16 行）。随后，应用基于相异差键过滤策略生成候选集，并验证候选集来获得最终的结果（第 17~28 行）。最后，返回结果集（第 29 行）。

算法 4：LMB-AD

输入： Q：查询集合

δ：相似度阈值

输出： RS：数据集 D 中与查询集合 Q 满足相似度阈值 δ 的所有集合

1：$RS \leftarrow NULL$

2：$L_- \leftarrow \lceil |Q| * \delta \rceil, L_+ \leftarrow \lfloor |Q| / \delta \rfloor$

3：if $L_- > (2^b - 1)$ then

4：　append each $S \in D$ with $|S| \in [L_-, L_+]$ into RS if $J(S,Q)$ is $True$

5：　return RS

6：if $L_+ > (2^b - 1)$ then

7：　append each $S \in D$ with $|S| \in [2^b, L_+]$ into RS if $J(S,Q)$ is $True$

8：　$\max L \leftarrow 2^b - 1$

9：else

10：　$\max L \leftarrow L_+$

11：foreach L in $[L_-, \max L]$ do

12：　if $LPI[L]$ is $NULL$ then

13：　　continue

14：　$B_L \leftarrow LPI[L]$

15：　$\nabla_{L,Q} \leftarrow \dfrac{|Q| - L * \delta}{1 + \delta}, \Delta_{L,Q} \leftarrow \dfrac{L - |Q| * \delta}{1 + \delta}$

16：　$K_{L,-} \leftarrow GetLBKey(Q, \nabla_{L,Q}), K_{L,+} \leftarrow GetUBKey(Q, \Delta_{L,Q})$

17：　foreach $K \in [K_{L,-}, K_{L,+}]$ indexed in B_L do

18：　　$sum1 \leftarrow 0, sum2 \leftarrow 0, flag \leftarrow True$

19：　　for $i \leftarrow 1$ to M do

20：　　　if $Q.v_i \geq K.v_i$ then

21：　　　　$sum1 \leftarrow sum1 + Q.v_i - K.v_i$

22：　　　else

23：　　　　$sum2 \leftarrow sum2 + K.v_i - Q.v_i$

24：　　　if $sum1 > \nabla_{L,Q}$ or $sum2 > \Delta_{L,Q}$ then

25：　　　　$flag \leftarrow False$

26：　　　　break

27：　　if $flag$ is $True$ then

28：　　　append each $S \in K.Vec$ into RS if $J(S,Q)$ is $True$

29：return RS

示例 4.2：给定一个查询集合 $Q=\{e_1,e_2,e_6,e_7,e_{11},e_{12},e_{16},e_{20}\}$ 和相似度阈值 $\delta=0.7$。通过键生成策略可以得到 $Q.K=[2,2,2,2]$。根据长度过滤可以得到长度下界 $L_-=6$ 和长度上界 $L_+=11$。当长度 $L=6$ 时，可以得到 $\nabla_{6,Q}=2$ 和 $\Delta_{6,Q}=0$。通过 FBFA 算法，可以得到键下界 $K_{6,-}=[0,2,2,2]$ 和键上界 $K_{6,+}=[2,2,2,2]$。表 4.1 展示了每个长度 $L\in[L_-,L_+]$ 下的对称差上界 $\Lambda_{L,Q}$、上界差上界 $\nabla_{L,Q}$、下界差上界 $\Delta_{L,Q}$、以及分别基于对称差上界和相异差上界生成的键下界 $K_{L,-}$ 和键上界 $K_{L,+}$ 的实际计算结果。

表 4.1 基于对称差的键边界与基于相异差的键边界对比

L	$\Lambda_{L,Q}$	$K_{L,-}$	$K_{L,+}$	$\nabla_{L,Q}$	$\Delta_{L,Q}$	$K_{L,-}$	$K_{L,+}$
6	2	[0,2,2,2]	[4,2,2,2]	2	0	[0,2,2,2]	[2,2,2,2]
7	2	[0,2,2,2]	[4,2,2,2]	1	0	[1,2,2,2]	[2,2,2,2]
8	2	[0,2,2,2]	[4,2,2,2]	1	1	[1,2,2,2]	[3,2,2,2]
9	3	[0,1,2,2]	[5,2,2,2]	1	2	[1,2,2,2]	[4,2,2,2]
10	3	[0,1,2,2]	[5,2,2,2]	0	2	[2,2,2,2]	[4,2,2,2]
11	3	[0,1,2,2]	[5,2,2,2]	0	3	[2,2,2,2]	[5,2,2,2]

由表 4.1 可知，基于相异差生成的键边界比基于对称差生成的键边界小得多。

4.5 复杂度分析

4.5.1 空间复杂度

LMB 索引：跟第三章的分析相同，这里就不赘述了。LMB 的总空间开销为：

$(2N-P)X+\frac{f(N-P)+N}{f-1}Y+|D|*Z+(2^b-1)*Y$。

LMB-AD 算法：通过分析算法 4 给出了 LMB-AD 算法的代码，可以发现，LMB-AD 算法只使用了常数级别的空间，因此空间复杂度为 $O(1)$。

4.5.2 时间复杂度

假设长度上界 $L_+\le(2^b-1)$，即查询都在 LMB 索引中执行，设 $\mathrm{A}=L_+-L_-$，执行 A 次 FPFA 策略生成键边界的时间开销 $O(AM)$。在 LMB 中查找键下界 $K_{L,-}$ 的时间复杂度为 $O(A\log_f(N/P))$。键过滤的时间成本为 $O(N_1^{\#}*M)$，其中，$N_1^{\#}$ 是位于键边界 $[K_{L,-},K_{L,+}]$ 内的键数量，验证的时间开销为 $O(N_2*V^{\#})$，其中，N_2 是候选集合的数量，$V^{\#}$ 是采用基于最小覆盖和提前中止的验证策略验证两个集合

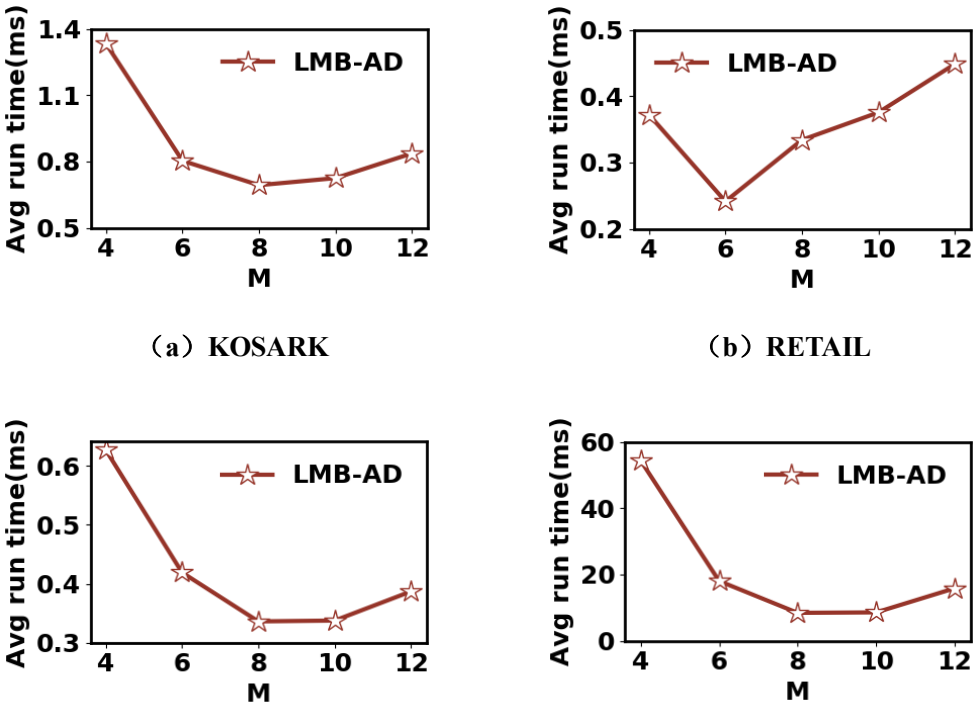
的平均成本。因此，LMB-AD 算法的时间复杂度为 $O(AM+A\log_f(N/P)+N_1^{\#}*M+N_2*V^{\#})$。与 LMB-SD 的相比，$N_1^{\#}<N_1$，$V^{\#}<V$，因此 LMB-AD 的时间复杂度更低。

4.6 实验分析

本章的实验环境、数据集和第三章的相同，这里就不赘述了。

4.6.1 分区数对查询效率的影响

为了评估分区数 M 对 LMB-AD 查询效率的影响，本节固定相似度阈值 δ =0.7，并将分区数 M 从 4 变化到 12。实验结果如图 4.3 所示。



（a）KOSARK　　（b）RETAIL

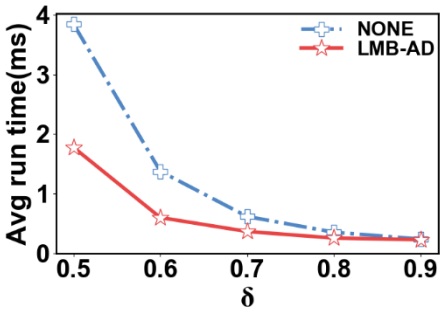
（c）LASTFM　　（d）TWEETS

图 4.3 分区数对查询效率的影响

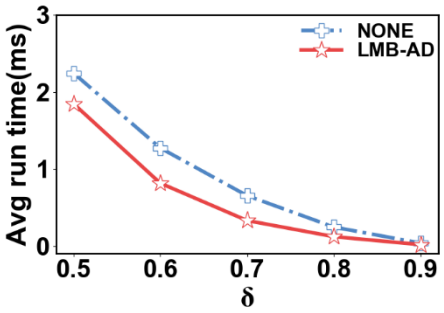
从图 4.3 中可以观察到，LMB-AD 的变化规律和 LMB-SD 的相同，这里就不赘述了。考虑到在四个数据集上，LMB-AD 的性能在 M =8 时接近最佳，本章在后续实验中采用此作为默认的分区数。

4.6.2 过滤策略的效率

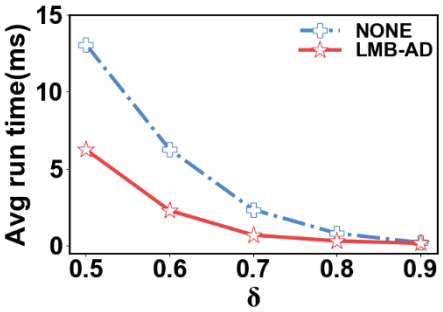
为了评估过滤策略的效率，本节在 LMB-AD 算法中删除了基于相异差的键过滤策略，记为 NONE。平均查询时间的对比结果如图 4.4 所示。



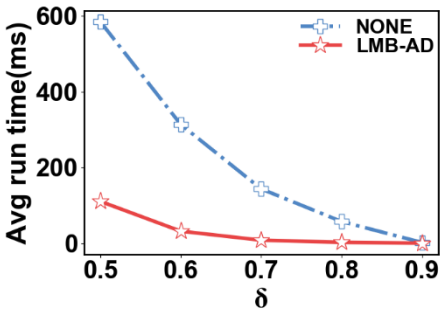
（a）KOSARK



（b）RETAIL

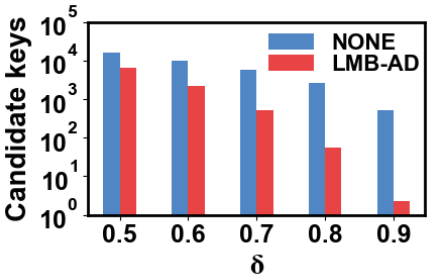


（c）LASTFM

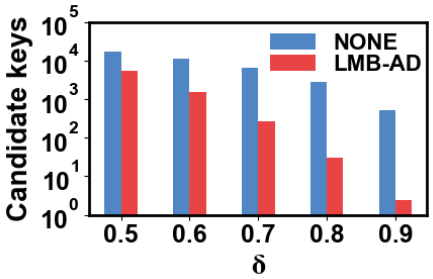


（d）TWEETS

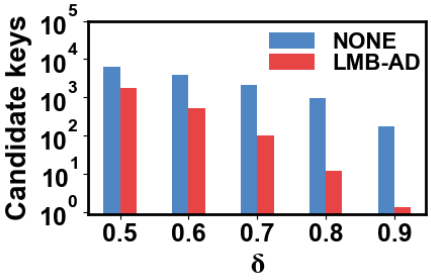
图 4.4 过滤策略的效率



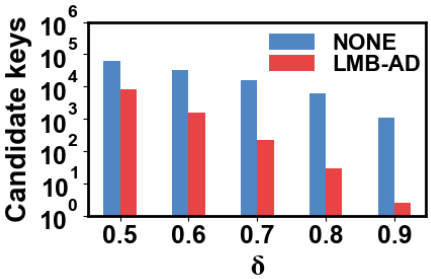
（a）KOSARK



（b）RETAIL



（c）LASTFM



（d）TWEETS

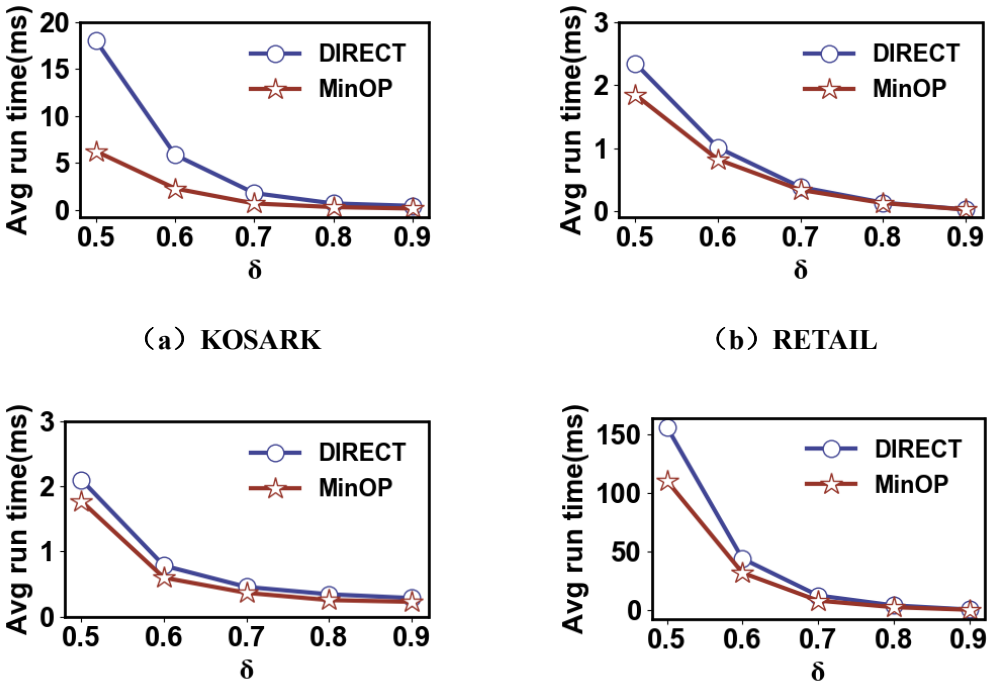
图 4.5 过滤策略的候选键数量

从图 4.4 中可以看出，基于相异差的键过滤策略，LMB-AD 算法始终由于 NONE 算法。具体而言，在 KOSARAK，RETAIL，LASTFM 和 TWEETS 上，LMB-AD 实现的效率提升高达 2.38 倍，0.99 倍，1.30 倍和 19.24 倍。

为了进一步解释基于相异差的键过滤策略的优秀性能，图 4.5 对比了 LMB-AD 和 NONE 的候选键数量（这里是指经过键过滤之后得到的候选键的规模）。从图 4.5 中可以看出，基于相异差的键过滤策略可以显著降低候选键的规模。

4.6.3 验证策略的有效性

为了评估验证策略的有效性，本章在对比了两种验证策略：DIRECT 和 MinOP。DIRECT 是指传统的基于直接验证的策略，而 MinOP 指的是基于最小覆盖的提前中止的策略。图 4.6 展示了这两种策略的实验结果。



（a）KOSARK　　（b）RETAIL

（c）LASTFM　　（d）TWEETS

图 4.6 不同验证策略下的平均查询时间

由图 4.6 可见，得益于 MinOP 的最小覆盖和提前中止，在所有四个数据集上，MinOP 的效果均优于 DIRECT。在 KOSARAK，RETAIL，LASTFM 和 TWEETS 上，MinOP 相对 DIRECT 的效率提升最高可分别达到 65.3%，21.3%，25.7%和 35.1%。

4.6.4 对比键边界计算策略

本节比较了三种不同的键边界计算策略：LF，SD 和 AD。LF 表示仅使用长度过滤的键边界计算策略，SD 表示结合长度过滤和对称差的键边界计算策略，而 AD 表示结合长度过滤和相异差的键边界计算策略。对比这三种键边界计算策略生成候选键的数量，实验结果如图 4.7 所示。

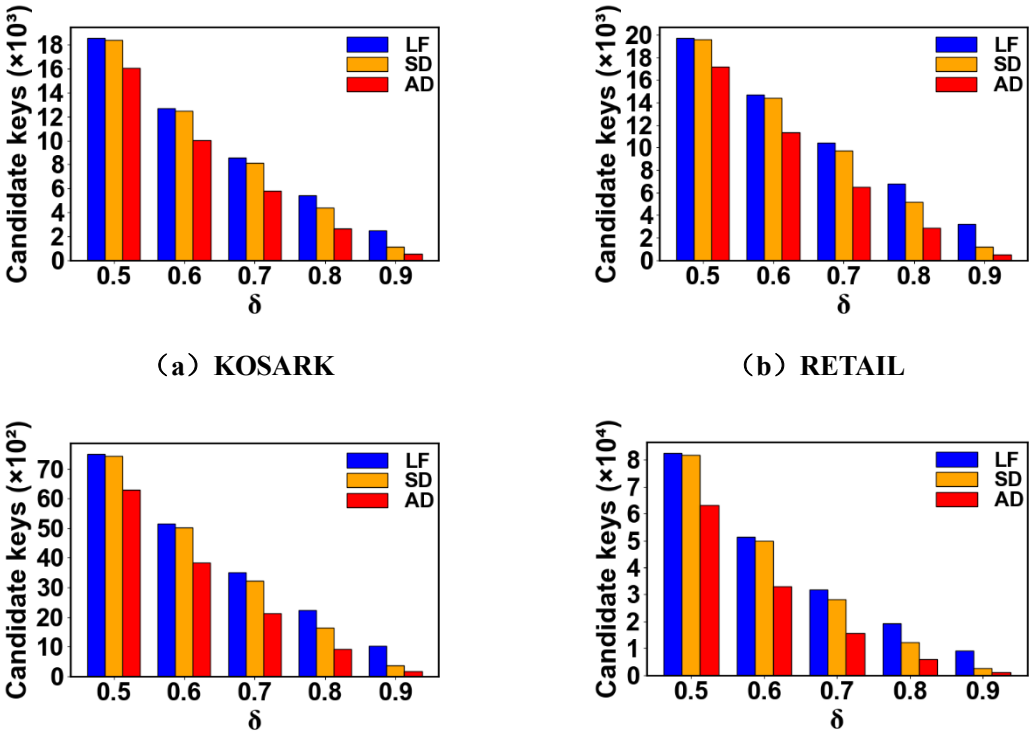


图 4.7 不同键边界计算策略下的候选键数量

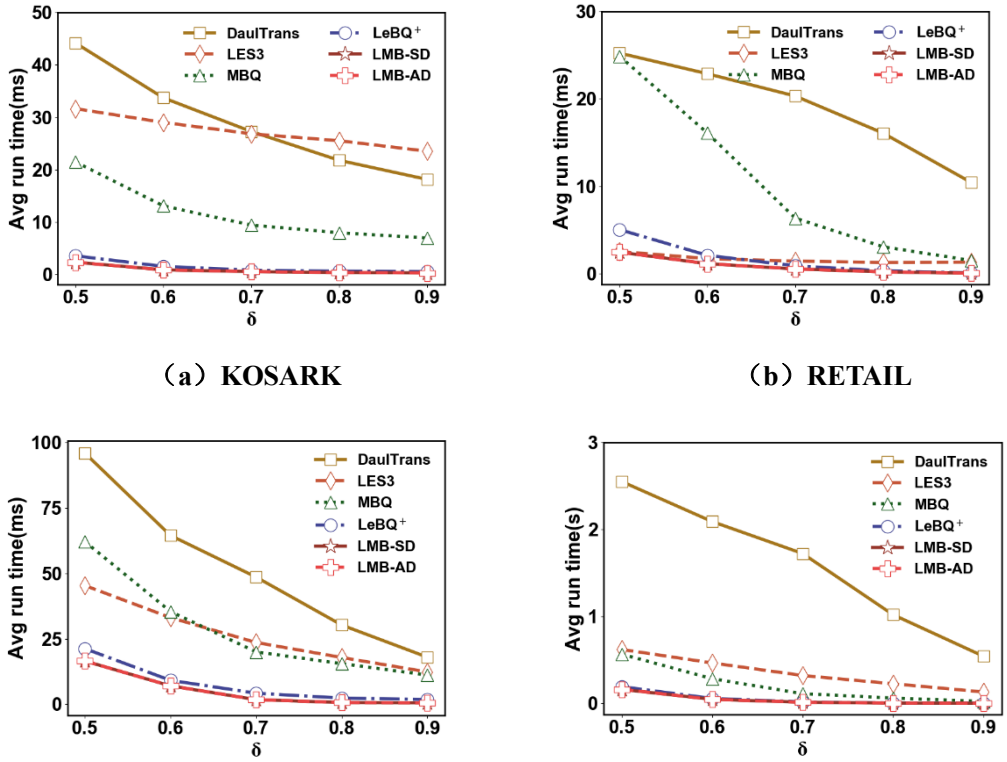
如图 4.7 所示，AD 的性能明显优于其他两种，它生成了最少了候选键。此外，随着相似度阈值δ的增加，AD 优势继续扩大。对于 KOSARAK，当相似度阈值δ=0.9 时，AD 平均每个查询仅返回 519.16 个键，仅为 SD 的 46%和 LF 的 21%。

4.6.5 对比其他算法

本节将 LMB-AD 与其他集合相似度查询算法进行了比较，包括 MBQ，DualTrans，LES3、LeBQ^{+}以及第三章提出的 LMB-SD 算法，结果如图 4.8 所示。

如图 4.8 所示，LMB-AD 的查询效率明显优于其他算法。当相似度阈值δ=0.9 时，在 KOSARAK，RETAIL，LASTFM 和 TWEETS 上，LMB-AD 的查询执行时间分别为 0.15ms、0.02ms、0.22ms 和 0.67ms，它的查询效率比 MBQ 提高

了 75 倍，比 LeBQ⁺提高了 11.13 倍。对于第三章提出的 LMB-SD 算法，效率提升了 2.87 倍。



（a）KOSARK　　（b）RETAIL

（c）LASTFM　　（d）TWEETS

图 4.8 不同算法的平均查询时间

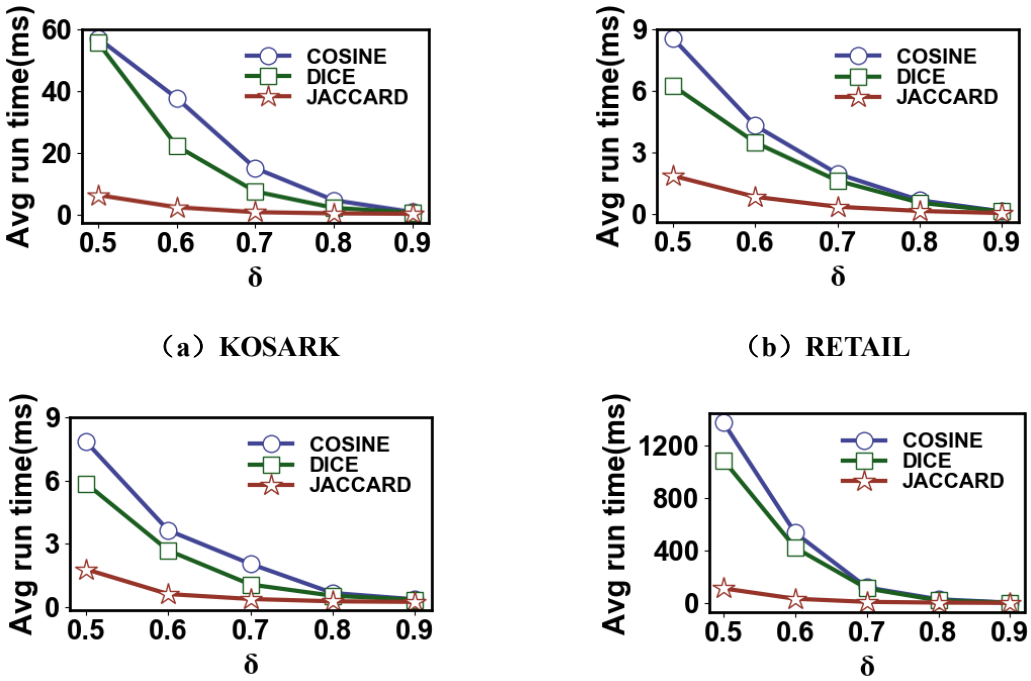
4.6.6 扩展到其他相似度函数

LMB-SD 算法支持扩展到 Cosine、Dice 等其他相似度函数。本章提出的 LMB-AD 算法同样支持扩展到其他的相似度函数，扩展到这些函数的关键之处在于计算对应的长度下界 L_-、长度上界 L_+ 以及查询集合 Q 长度特定 L 的相异差上界 $\nabla_{L,Q}$ 和 $\Delta_{L,Q}$，如表 4.2 所示。

表 4.2 LMB-AD 扩展到其他相似度函数

相似度函数	L_-	L_+	$\nabla_{L,Q}$	$\Delta_{L,Q}$
Jaccard	$\lceil \lvert Q\rvert * \delta \rceil$	$\left\lfloor \frac{\lvert Q\rvert}{\delta} \right\rfloor$	$\left\lfloor \frac{\lvert Q\rvert - L*\delta}{1+\delta} \right\rfloor$	$\left\lfloor \frac{L-\lvert Q\rvert*\delta}{1+\delta} \right\rfloor$
Cosine	$\lceil \lvert Q\rvert * \delta^2 \rceil$	$\left\lfloor \frac{\lvert Q\rvert}{\delta^2} \right\rfloor$	$\left\lfloor \lvert Q\rvert - \delta\sqrt{L\lvert Q\rvert} \right\rfloor$	$\left\lfloor L - \delta\sqrt{L\lvert Q\rvert} \right\rfloor$
Dice	$\left\lceil \lvert Q\rvert * \frac{\delta}{2-\delta} \right\rceil$	$\left\lfloor \frac{2-\delta}{\delta}\lvert Q\rvert \right\rfloor$	$\left\lfloor \lvert Q\rvert - \frac{\delta}{2}(L+\lvert Q\rvert) \right\rfloor$	$\left\lfloor L - \frac{\delta}{2}(L+\lvert Q\rvert) \right\rfloor$

为了评估 LMB-AD 算法在不同相似度函数下的性能，本节使用 Jaccard、Cosine 和 Dice 函数分别进行实验。实验对比结果如图 4.9 所示。在所有数据集中，随着相似度阈值δ的增加，每个相似度函数生成的候选集都会减少，从而减少运行时间。其中，Jaccard 生成的候选集最少，运行时间最短。



（a）KOSARK　　（b）RETAIL

（c）LASTFM　　（d）TWEETS

图 4.9 不同相似度函数下的平均查询时间

4.7 本章小结

针对 LMB-SD 中基于对称差所生成键边界仍相对宽泛、候选键集规模较大的问题，本章进一步引入相异差思想，将原有的整体差异拆分为两个单向差，并在此基础上提出了基于相异差的集合相似度查询算法 LMB-AD。在保持 LMB 索引结构不变的基础上，LMB-AD 通过基于长度过滤和相异差的键边界计算策略生成更加紧凑的键边界，并结合基于相异差的键过滤策略进一步减少候选键集规模。此外，本章还引入了基于最小覆盖和提前中止的验证策略，以降低验证阶段的计算开销，从而进一步提升整体查询效率。最后，通过对比实验与结果分析，验证了 LMB-AD 的有效性。实验结果表明，LMB-AD 在查询性能上优于现有对比方法，同时也优于上一章提出的 LMB-SD 算法。从计算机系统结构的角度分析，LMB-AD 通过相异差实现了更细粒度的差异建模，将键边界范围缩小近一半，有效减少了无效索引扫描与候选键集规模，从查询执行路径层面进一步提升

了系统效率。

5 总结与展望

5.1 本文主要工作总结

集合相似度查询在文本检索、网页去重、数据清洗和生物信息分析等领域具有广泛应用价值。作为计算机系统结构领域的重要研究方向，如何设计高效的索引结构与查询处理机制以支持海量集合数据上的实时相似度检索，对于提升系统整体性能具有重要意义。然而，现有方法普遍存在集合长度信息利用不足、候选控制能力有限以及过滤阶段计算代价较高等问题，从而制约了查询效率的进一步提升。针对上述问题，本文围绕集合相似度查询中的索引组织优化、搜索边界收缩以及候选与验证开销控制开展研究，主要完成了以下工作：

首先，本文设计了一种基于长度分区的多 B^+树索引结构 LMB，并在此基础上提出了一种基于对称差的集合相似度查询算法 LMB-SD。LMB 首先按照集合长度对数据集进行分区，将具有相同长度的集合划分到同一分区中；随后将集合映射为一维整型键，并为每个长度分区分别构建独立的 B^+子树；同时，引入长度分区索引 LPI 对这些 B^+子树进行统一管理，从而形成高效的二级索引结构。通过将整个数据集组织为多个独立的局部索引，从而降低单一索引在大规模场景下的搜索开销。基于 LMB 索引，LMB-SD 在查询阶段首先利用长度过滤进行初步筛选，然后基于长度过滤和对称差快速生成长度特定的键边界，避免了传统方法中逐节点计算相似度上界所带来的高昂开销；进一步地，算法采用基于对称差的键过滤策略，对位于键边界范围内的键进行深度过滤，生成候选集，并通过精确验证得到最终结果。四个真实数据集上的实验结果表明，LMB-SD 的查询性能显著优于对比方法，较 MBQ 最高提升 29.40 倍，较 $LeBQ^+$最高提升 2.14 倍。

其次，为进一步压缩搜索边界、减少候选键集的规模，本文在 LMB 索引基础上提出了一种基于相异差的集合相似度查询算法 LMB-AD。该算法与 LMB-SD 采用相同的索引结构和基本查询框架，其核心改进在于将对称差拆分为两个相异的单向差，并基于相异差设计新的键边界计算策略，从而生成更加紧凑的键边界。相应地，本文进一步设计了基于相异差的键过滤策略进行深度过滤，并结合基于最小覆盖和提前中止的验证策略，以降低后续验证开销。实验结果表明，LMB-AD 在整体性能上优于 LMB-SD，查询效率较 LMB-SD 最高提升 2.87 倍，验证了该方法的有效性。

本文围绕集合相似度查询中“长度信息利用不足、搜索边界较宽以及过滤与

查询效率不高”等问题，提出了基于长度分区多 B^+树索引的两种高效查询算法。研究结果表明，所提方法能够有效缩小搜索范围、减少候选集规模并提升查询效率。本文所提算法具备良好的实际应用价值。LMB-SD 与 LMB-AD 可对接文本检索、电商去重、社交网络分析、生物信息比对等典型集合数据处理场景。此外，本文所提出的索引组织与查询优化方法，体现了计算机系统结构专业“面向应用负载设计高效系统”的核心研究范式。

5.2 本文进一步的研究方向

尽管本文提出的 LMB-SD 和 LMB-AD 算法取得了较好的查询性能，但仍存在一定的局限性。未来的研究可从以下几个方向进一步展开：

（1）基于学习模型的参数自适应优化

在 LMB、LMB-SD 和 LMB-AD 中，分区数 M 会对索引构建质量和查询性能产生显著影响。目前，最优分区值 M 主要通过实验确定，自动化程度仍然有限。未来可引入机器学习或代价模型方法，结合数据集特征，如集合平均长度、长度分布、元素频率分布和数据规模等，建立数据特征与最优分区参数 M 之间的映射关系，实现参数的自适应推荐与动态调整，从而提高算法的鲁棒性和可迁移性。

（2）多维特征融合的混合分区策略

本文采用的长度分区策略简单高效，但本质上仅利用了集合长度这一单一特征。未来可进一步研究融合多维特征的混合分区方法，例如结合集合长度、元素分布特征、局部敏感哈希签名或语义特征表示，对数据进行更加细粒度的划分。通过提高分区内部对象的相似潜力、降低跨分区搜索频率，有望进一步增强候选控制能力，在更加复杂的数据分布场景下提升查询性能。

（3）复杂数据类型的泛化

本文当前主要针对整型元素构成的集合数据进行研究与实验验证，但所提出的方法框架并不局限于整型数据本身，而是依赖于数据对象能够被表示为集合并进一步映射为适合索引组织的离散特征。对于浮点数、字符串、文本、日期时间以及音频视频等更复杂的数据类型，虽然需要额外设计离散化、编码映射、特征提取或语义表示机制，并可能引入精度控制、验证逻辑复杂化等问题，但这些问题并非不可解决。未来可围绕不同数据类型的特征表示方式、映射机制和相应验证策略开展研究。

（4）低阈值场景下的查询优化机制

在相似度阈值较低时，候选集规模往往急剧膨胀，导致过滤和验证开销显著增加，从而使查询性能明显下降。针对这一问题，后续可探索多阶段级联过滤机制。例如，在现有算法基础上引入轻量级签名过滤，如 MinHash 或 SimHash，作为前置近似过滤器，快速剔除大部分明显不相似的集合；或者设计面向低阈值场景的并行验证机制，利用多核 CPU 或 GPU 的并行计算能力加速大规模候选集验证，从而在保证查询精度的前提下突破性能瓶颈。

致谢

白驹过隙，岁月流金。三年的研究生求学路转瞬即逝，此刻驻足回望，往昔奋斗的点滴历历在目。从最初在文献海洋中茫然探索，到确立研究方向、搭建实验框架，再到论文的撰写、修改与最终定稿，这一路走来，既有山重水复的困顿与迷茫，亦有柳暗花明的豁然与喜悦。这些磨砺不仅拓宽了我的学术视野，更锤炼了我的心性，成为我人生中一笔宝贵的财富。这篇论文的完成，离不开师长、亲友及同窗的关怀与支持，在此，我愿以此文略表寸心。

首先，我要向我的恩师贾连印老师致以最崇高的敬意与最深切的感激。师恩如海，衔草难报。在入学之初，面对陌生的研究领域，是贾老师不辞辛劳，通过一对一的悉心讲解，为我剖析“集合相似度查询”的核心逻辑，并指点文献阅读的门径，助我快速叩开了学术的大门。在课题研究受阻、实验数据不尽人意之时，老师总能一针见血地指出症结，给予我耐心的分析与鼓励。尤其令我动容的是，从论文初稿的雏形到最终稿的定论，老师不厌其烦地为我进行了十多轮的细致批注与斧正。记得在投稿会议论文期间，为了赶上截稿日期，老师曾多次陪我奋战至凌晨一两点，反复推敲每一个细节。贾老师严谨治学的态度与废寝忘食的精神，将深深影响我未来的处世之道。在此，祝愿贾老师学术之树常青，桃李满天下。

游子远行，父母恩重。感谢生我养我的父母，你们是我身后最坚实的避风港。求学在外，是你们无条件的信任与包容，消解了我的后顾之忧；无论遭遇何种挫折，家永远是我汲取力量的源泉。这份深沉的爱，是我不断前行的最大动力。同时，也要感谢我的挚友们，是你们的欢声笑语驱散了科研的枯燥，感谢你们多年来的陪伴与理解。未来的日子里，愿我们都能不负韶华，我亦将以不懈的努力来回报这份深情。

此外，感谢母校为我提供了优秀的科研环境与学习平台，让我得以心无旁骛地求索真知；感谢各位授课老师的传道授业，夯实了我的理论基础。特别感谢实验室的苗师兄，在我科研陷入瓶颈时，你们不吝赐教，提供了许多宝贵的建议，令我受益良多；感谢实验室的同门师兄弟姐妹以及朝夕相处的室友们，正是有你们在生活上的互助与学习上的切磋，这三年的时光才显得如此鲜活而热烈。

凡是过往，皆为序章。研究生的终点，亦是人生的新起点。带着这份沉甸甸的收获与祝福，我将以更加饱满的姿态奔赴下一场山海。衷心祝愿各位师长、同窗前程似锦，万事顺遂，在未来的广阔天地中书写属于自己的华章。

参考文献

[1] Jia L Y, Li S S, Zhang Y N, et al. Trie and LOUDS hybrid model for efficient e-commerce processing in cloud environment[J]. Simulation Modelling Practice and Theory, 2024, 134: 102960.

[2] Roy I, Agarwal R, Chakrabarti S, et al. Locality sensitive hashing in Fourier frequency domain for soft set containment search[C]//Proceedings of the 37th International Conference on Neural Information Processing Systems. Red Hook, NY: Curran Associates Inc., 2023: 56352-56383.

[3] Widmoser M, Kocher D, Augsten N, et al. MetricJoin: Leveraging metric properties for robust exact set similarity joins[C]//2023 IEEE 39th International Conference on Data Engineering (ICDE). Anaheim, CA, USA: IEEE, 2023: 1045-1058.

[4] Schmitt D, Kocher D, Augsten N, et al. A two-level signature scheme for stable set similarity joins[J]. Proc. VLDB Endow., 2023, 16(11): 2686-2698.

[5] Li M, Jia L, Hu J, et al. An efficient length-segmented inverted index-based set similarity query algorithm[J]. Int. J. Comput. Sci. Math., 2022, 16(1): 85-95.

[6] Jia L, Luo S, Ding J, et al. A length enhanced B^{+}-tree based index for efficient set similarity query[C]//2025 IEEE 41st International Conference on Data Engineering (ICDE). Hong Kong: IEEE, 2025: 1292-1304.

[7] Zhang Y, Wu J, Wang J. A transformation-based framework for KNN set similarity search[J]. IEEE Transactions on Knowledge and Data Engineering, 2018, 32(3): 409-423.

[8] 李莉, 杨春艳, 朱江文. 区块链下社交网络用户抄袭识别方案[J]. 计算机应用, 2024, 44(01): 242-251.

[9] Chang C Y, Lee S J, Wu C H. Using word semantic concepts for plagiarism detection in text documents[J]. Information Retrieval Journal, 2021, 24: 298-321.

[10] Karnalim O, Simon, Chivers W. Educating students about programming plagiarism and collusion via formative feedback[J]. ACM Transactions on Computing Education, 2022, 22(3): 1-31.

[11] Mansoor M N, Al-Tamimi M S H. Computer based plagiarism detection techniques:

A comparative study[J]. International Journal of Nonlinear Analysis and Applications, 2022, 13(1): 3599-3611.

[12] Mathew G, Stolee K T. Cross language code search using static and dynamic analyses[C]//Proceedings of the 29th Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021: 205-217.

[13] 张峰, 韦友良, 秦玉成. 基于程序流程图和图注意力网络的跨语言代码抄袭检测方法[J]. 小型微型计算机系统, 2025, 46(01): 249-256.

[14] 赵文涛, 冯婷婷. 面向稀疏数据的协同过滤算法相似度[J]. 计算机工程与设计, 2024, 45(06): 1728-1734.

[15] 张茂宇, 李海明. 融合双重语义相似度的混合协同过滤推荐算法[J]. 计算机工程与设计, 2023, 44(09): 2671-2678.

[16] 随晓文. 基于标签相似度的数字化书籍资源智能推荐算法[J]. 吉林大学学报(信息科学版), 2024, 42(03): 516-521.

[17] 贾凡, 康舒雅, 江为强. 基于相似性的多用户漏洞推荐算法[J]. 清华大学学报(自然科学版), 2023, 63(09): 1399-1407.

[18] 车翔玖, 李良. 融合全局与局部细粒度特征的图相似度度量算法[J]. 吉林大学学报(工学版), 2025, 55(07): 2365-2371.

[19] Ling X, Wu L, Wang S. Multilevel graph matching networks for deep graph similarity learning[J]. IEEE Transactions on Neural Networks and Learning Systems, 2023, 34: 799-813.

[20] Ranjan R, Grover S, Medya S. Greed: A neural framework for learning graph distance functions[J]. Advances in Neural Information Processing Systems, 2022, 35: 22518-22530.

[21] Zhuo W, Tan G. Efficient graph similarity computation with alignment regularization[J]. Advances in Neural Information Processing Systems, 2022, 35: 30181-30193.

[22] Tai D, Fang J. SymDex: Increasing the efficiency of chemical fingerprint similarity searches for comparing large chemical libraries by using query set indexing[J]. Journal of Chemical Information and Modeling, 2012, 52(8): 1926-1935.

[23] Qi S Y, Bouros P, Mamoulis N. Top-k string similarity joins[C]//32nd International

Conference on Scientific and Statistical Database Management (SSDBM), Vienna: ACM, 2020: 1-12.

[24] Sandes E F, Teodoro G L, Melo A C. Bitmap filter: Speeding up exact set similarity joins with bitwise operations[J]. Information Systems, 2020, 88: 101-449.

[25] 黄志鸿. 基于 FP-tree 和 MapReduce 的集合相似性连接[D]. 深圳: 深圳大学, 2020.

[26] 冯禹洪, 吴坤汉, 黄志鸿. 基于 FP-tree 和 MapReduce 的集合相似度自连接算法[J]. 计算机研究与发展, 2023, 60(12): 2890-2906.

[27] Wang J N, Li G L, Feng J H. Can we beat the prefix filtering?: an adaptive framework for similarity join and search[C]//Proceedings of the ACM SIGMOD International Conference on Management of Data. Scottsdale: ACM, 2012: 85-96.

[28] Xiao C, Wang W, Lin X M. Efficient similarity joins for near-duplicate detection[J]. ACM Transactions on Database Systems, 2011, 36(3): 1-41.

[29] Rong C, Lu W, Wang X, et al. Efficient and scalable processing of string similarity join[J]. IEEE Transactions on Knowledge and Data Engineering, 2012, 25(10): 2217-2230.

[30] Rong C, Lin C, Silva Y N. Fast and scalable distributed set similarity joins for big data analytics[C]//2017 IEEE 33rd International Conference on Data Engineering (ICDE). San Diego: IEEE, 2017: 1059-1070.

[31] Bellas C, Gounaris A. An empirical evaluation of exact set similarity join techniques using GPUs[J]. Information Systems, 2020, 89: 101-485.

[32] Gallet B, Gowanlock M. Load Imbalance Mitigation Optimizations for GPU-Accelerated Similarity Joins[C]//2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). Rio de Janeiro: IEEE, 2019: 396-405.

[33] Feng Y, Tang J, Liu M. Fast Document Cosine Similarity Self-Join on GPUs[C]//2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI). Volos: IEEE, 2018: 205-212.

[34] Yu C, Nutanong S, Li H. A generic method for accelerating LSH-based similarity join processing[J]. IEEE Transactions on Knowledge and Data Engineering, 2016, 29(4): 712-726.

[35] Christiani T, Pagh R, Sivertsen J. Scalable and robust set similarity join[C]//2018 IEEE 34th International Conference on Data Engineering (ICDE). Paris: IEEE, 2018: 1240-1243.

[36] Sohrabi M K, Azgomi H. Parallel set similarity join on big data based on Locality-Sensitive Hashing[J]. Science of Computer Programming, 2017, 145: 1-12.

[37] Florian H, Nicolas L, Florian K. Joins Over Encrypted Data with Fine Granular Security[C]//2019 IEEE 35th International Conference on Data Engineering (ICDE). Macao: IEEE, 2019: 674-685.

[38] Zeakis A, Skoutas D, Sacharidis D, et al. Tokenjoin: efficient filtering for set similarity join with maximum weighted bipartite matching[J]. Proceedings of the VLDB Endowment, 2022, 16(4): 790-802.

[39] Yang Z, Zheng B, Li G, et al. Adaptive top-k overlap set similarity joins[C]//2020 IEEE 36th International Conference on Data Engineering (ICDE). Dallas: IEEE, 2020: 1081-1092.

[40] Wang Z, Zuo C, Deng D. Txtalign: efficient near-duplicate text alignment search via bottom-k sketches for plagiarism detection[C]//Proceedings of the 2022 International Conference on Management of Data. Philadelphia: ACM, 2022: 1146-1159.

[41] Wang Z, Li Q, Zhang G, et al. Exploring set similarity for dense self-supervised representation learning[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. New Orleans: IEEE, 2022: 16590-16599.

[42] Yang J, Zhang W, Wang X, et al. Distributed streaming set similarity join[C]//2020 IEEE 36th International Conference on Data Engineering (ICDE). Dallas: IEEE, 2020: 565-576.

[43] Chen L, Gao Y, Song X, et al. Indexing metric spaces for exact similarity search[J]. ACM Computing Surveys, 2022, 55(6): 1-39.

[44] Jiang Y, Deng D, Wang J. Efficient parallel partition-based algorithms for similarity search and join with edit distance constraints[C]//Proceedings of the Joint EDBT/ICDT 2013 Workshops. Genoa: ACM, 2013: 341-348.

[45] Cong G, Jensen C S, Wu D M. Efficient retrieval of the top-k most relevant spatial

web objects[J]. Proceedings of the VLDB Endowment, 2009, 2(1): 337-348.

[46] Li Z S, Lee K C K, Zheng B H, et al. IR-tree: an efficient index for geographic document search[J]. IEEE Transactions on Knowledge and Data Engineering, 2011, 23(4): 585-599.

[47] Zhang Y, Li X X, Wang J, et al. An efficient framework for exact set similarity search using tree structure indexes[C]//2017 IEEE 33rd International Conference on Data Engineering (ICDE). San Diego: IEEE, 2017: 759-770.

[48] Guan Y G, Lu R X, Zheng Y D. EPGQ: Efficient and private feature-based group nearest neighbor query over road networks[J]. IEEE Internet of Things Journal, 2022, 9(20): 20492-20502.

[49] Jia L, Xi J, Li M, et al. ETI: an efficient index for set similarity queries[J]. Frontiers of Computer Science, 2012, 6(6): 700-712.

[50] Jia L, Tang J, Li M, et al. A trie based set similarity query algorithm[J]. Mathematics, 2023, 11(1): 229.

[51] Li Y F, Yu X H, Koudas N. LES3: learning-based exact set similarity search[J]. Proceedings of the VLDB Endowment, 2021, 14(11): 2073-2086.

[52] Chauhan S S, Batra S. A parallel computational approach for similarity search using Bloom filters[J]. Computational Intelligence, 2018, 34(2): 713-733.

[53] Deng D, Li G L, Feng J H. A pivotal prefix based filtering algorithm for string similarity search[C]//International Conference on Management of Data (SIGMOD). Snowbird: ACM, 2014: 673-684.

[54] Yang C, Chen L, Wang H, et al. Dynamic set similarity join: An update log based approach[J]. IEEE Transactions on Knowledge and Data Engineering, 2021, 35(4): 3727-3741.

[55] Zheng Y D, Lu R X, Guan Y G, et al. Achieving efficient and privacy-preserving exact set similarity search over encrypted data[J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(2): 1090-1103.

[56] Song F Y, Qin Z, Liu D X. Privacy-preserving task matching with threshold similarity search via vehicular crowdsourcing[J]. IEEE Transactions on Vehicular Technology, 2022, 70(7): 7161-7175.

[57] Mundra P, Zhang J, Nargesian F, et al. Koios: Top-k semantic overlap set

search[C]//2023 IEEE 39th International Conference on Data Engineering (ICDE). Anaheim, CA, USA: IEEE, 2023: 1531-1543.

[58] Fier F, Freytag J C. Parallelizing filter-and-verification based exact set similarity joins on multicores[J]. Information Systems, 2022, 108: 101912.

[59] Comer D. Ubiquitous b-tree[J]. ACM Comput. Surv., 1979, 11(2): 121-137.

[60] Manning C D, Raghavan P, Schütze H. Introduction to information retrieval[M]. Cambridge: Cambridge University Press, 2008.

[61] Guttman A. R-trees: A dynamic index structure for spatial searching[C]//Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data. Boston, Massachusetts: Association for Computing Machinery, 1984: 47-57.

[62] Fredkin E. Trie memory[J]. Communications of the ACM, 1960, 3(9): 490-499.

[63] Chan C-Y, Ioannidis Y E. Bitmap index design and evaluation[C]//Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data. Seattle, Washington, USA: Association for Computing Machinery, 1998: 355-366.

[64] Mann W, Augsten N, Bouros P. An empirical evaluation of set similarity join techniques[J]. Proc. VLDB Endow., 2016, 9(9): 636-647.

[65] Deng D, Li G, Wen H, et al. An efficient partition based method for exact set similarity joins[J]. Proc. VLDB Endow., 2015, 9(4): 360-371.

[66] Wang X, Qin L, Lin X, et al. Leveraging set relations in exact set similarity join[J]. Proc. VLDB Endow., 2017, 10(9): 925-936.

[67] Vernica R, Carey M J, Li C. Efficient parallel set-similarity joins using MapReduce[C]//Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data. Indianapolis, Indiana, USA: Association for Computing Machinery, 2010: 495-506.

[68] Arseneau Y, Gautam S, Nickerson B, et al. Stilt: Unifying spatial, temporal and textual search using a generalized multidimensional index[C]//Proceedings of the 32nd International Conference on Scientific and Statistical Database Management. ACM, 2020: 1–12.

附录 A 攻读硕士学位期间的学术成果

学术论文

[1] Lianyin Jia, Yongxue Zhao, Mengjuan Li, Xiuxing Li, Ying Jiang, and Jiaman Ding. An Asymmetric-difference based Document Exact Similarity Search Algorithm. In Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26).（CCF-A 类会议已录用）

[2] 赵永雪, 梁腾, 邢新明, 等. ABiGA: 面向中文多标签分类的轻量级模型[J]. 计算机与网络, 2025, 51(6): 541-547.（已见刊）

已受理发明专利

[1] 贾连印, 赵永雪, 李孟娟, 丁家满, 周海河, 曾瑞. 基于 B+树的长度独立的集合相似度查询方法及其应用.（已进入实质性审查阶段）

在读期间获奖情况

[1] 2024.12 第二十一届“华为杯”中国研究生数学建模竞赛国家二等奖

[2] 2024.11 二等学业奖学金

[3] 2025.12 三等学业奖学金

[4] 2023.09 优秀志愿者

附录 B 攻读硕士学位期间参与的科研项目

[1]